

Содержание

1	Введение	2
2	Обзор литературы	5
2.1	Системы интерактивного доказательства теорем	5
2.2	Методы оценивания автоматической формализации	6
2.3	Подходы к решению задачи автоматической формализации	10
2.4	Адаптация больших языковых моделей к задачи автоматической формализации	12
2.5	Ограничения существующих подходов.	16
3	Методы оценивания автоматической формализации	18
3.1	Постановка задачи	18
3.2	Данные для оценивания	19
3.3	Методы автоматической оценки на основе компилятора Lean 4	20
3.4	Применение больших языковых моделей к оцениванию	22
3.5	Результаты	24
4	Задача автоматической формализации	26
4.1	Постановка задачи	26
4.2	Данные для оценивания	26
4.3	Оценка существующих решений	27
4.4	Обучение модели	29
5	Заключение	35

1 Введение

Системы интерактивного доказательства теорем позволяют записывать математические утверждения и их доказательства на строго определённом формальном языке и механически проверять их корректность по правилам логики. В основе таких систем лежит соответствие Карри–Ховарда, отождествляющее высказывания с типами, а доказательства с термами соответствующего типа, в силу чего проверка доказательства сводится к алгоритмически разрешимой проверке типов, выполняемой надёжным ядром системы. За десятилетия развития были созданы зрелые ассистенты доказательств: Isabelle/HOL [1] и Coq [2], а в последние годы центральным инструментом для формализации современной математики стала система Lean 4 [3] вместе с крупнейшей библиотекой mathlib [4]. Ключевым свойством Lean 4 является то, что его компилятор выступает в роли надёжного оракула: он проверяет не только синтаксис записи, но и логическую корректность доказательства, давая точную и полностью автоматическую обратную связь о приемлемости формального текста.

Потребность в таких системах вызвана тем, что традиционная математическая строгость опирается на проверку доказательств людьми, которая трудоёмка и не застрахована от ошибок, тогда как современные доказательства достигают объёма и сложности, при которых ручная проверка становится практически невыполнимой. Системы интерактивного доказательства переносят эту проверку на ядро, гарантирующее, что ни один логический шаг не остаётся неоправданным, и тем самым сводят доверие ко всему доказательству к доверию небольшому проверенному ядру. Кроме того, накопленные знания собираются в переиспользуемые библиотеки формализованной математики, на которые можно строго опираться при доказательстве новых результатов, не воспроизводя их заново. Все это также упрощает командную работу над крупными доказательствами.

Несмотря на зрелость этих инструментов, их практическое применение остаётся крайне трудоёмким даже для опытных специалистов. Каждый очевидный для человека шаг рассуждения приходится развёртывать в несколько строк формального кода, при этом точность требуется на каждом логическом переходе. Это создаёт высокий барьер входа и резко ограничивает масштабы, в которых математика может быть формализована вручную. В результате громадный корпус математических результатов, записанных лишь на естественном языке, остаётся практически не связанным с инструментами интерактивного доказательства, а ручной перенос его в формальный вид требует усилий, несопоставимых с темпами роста самой математики.

Естественным способом снизить эту стоимость является автоматизация перевода математики на формальный язык. В последние годы произошёл качественный сдвиг, связанный с появлением больших языковых моделей: нейросетей на основе архитектуры трансформера [5], обучаемых на огромных текстовых корпусах и демонстрирующих способность порождать синтаксически и семантически корректный код. Показательна аналогия с обычным программированием: большие языковые модели уже существенно изменили процесс разработки программного обеспечения, став повседневным инструментом программиста. Поскольку формальное доказательство также является программой (терм соответствующего типа), естественно ожидать, что подобные модели окажутся полезны и для формальной математики. Существенно, что система интерактивного доказательства предоставляет то, чего лишено большинство задач генерации текста, а именно надёжный автоматический верификатор. Компилятор Lean 4 однозначно отклоняет некорректные построения, позволяя отделять правдоподобные, но ошибочные результаты модели от действительно верных.

Применение больших языковых моделей в формальной математике развивает-

ся по двум тесно связанным направлениям. Первое это автоматическое доказательство теорем, когда по уже формализованному утверждению модель строит его доказательство на Lean 4. И второе это автоматическая формализация, то есть перевод математических утверждений и доказательств с естественного языка на формальный. Именно второе направление снимает исходное узкое место, связывая обширный неформальный корпус с формальными инструментами и поставляя данные, на которых обучаются модели для доказательств.

Настоящая работа сосредоточена на более узкой, но базовой подзадаче: формализации только формулировок теорем, без их доказательств. Именно корректная формальная постановка является отправной точкой всего процесса. Бессмысленно доказывать формальное утверждение, если оно не соответствует исходному математическому смыслу. Кроме того, формализация формулировок напрямую влияет на обучение моделей для полного доказательства теорем. Современные модели для доказательств опираются на крупные корпуса формальных утверждений (miniF2F [6], ProofNet [7], FormalMATH [8]). Источником таких формальных утверждений и служит автоматическая формализация. Чем больше корректных формулировок удаётся получить из неформальных условий, тем шире становится обучающая и тестовая база для доказателей. При этом значимы не только объём, но и чистота этого корпуса. Ошибочные и неточные формализации привносят в обучающие данные шум, тогда как корректные формулировки служат для доказателя надёжной опорой. Эмпирически подтверждено, что обучение на автоматически формализованных утверждениях повышает качество доказателей. Дообучение на таких теоремах подняло долю доказанных задач miniF2F с 29,6 % до 35,2 % [9], а сочетание автоматической формализации с последующей фильтрацией позволило собрать около 8 млн пар «утверждение–доказательство» и обучить на них модель, достигшую 46,3 % на miniF2F [10]. Поэтому чем чище и качественнее формализованные утверждения, тем выше потолок качества обучаемой на этом корпусе модели доказательства теорем.

Основным препятствием на пути к надёжной автоматической формализации является отсутствие надёжного и автоматического способа оценить действительно ли полученная формализация соответствует исходному утверждению на естественном языке по смыслу. В отличие от доказательств, где успех однозначно подтверждается компилятором, при формализации компилятор проверяет лишь типовую корректность записи (typescheck), но не её смысловое соответствие. При этом экспертная оценка (ручная оценка людьми) точна, но дорога и не масштабируется. Тем самым надёжная автоматическая метрика семантической эквивалентности оказывается недостающим звеном как для измерения качества формализаторов, так и для их обучения.

Настоящая работа посвящена задаче автоматической формализации формулировок математических теорем на язык Lean 4 при помощи больших языковых моделей и охватывает оба связанных вопроса: как оценивать формализацию и как строить формализующую модель. Основной вклад работы заключается в следующем:

- Систематически сопоставлены автоматические методы оценивания формализации с экспертной разметкой.
- Предложен расширенный вариант метрики BEq [11], объединяющий двустороннюю проверку, расширенный набор семантически сохраняющих преобразований и предварительную нормализацию. Он повышает полноту с 32,7 % до 58,6 % и F_1 с 49,3 % до 73,8 % при сохранении почти нулевой доли ложноположительных срабатываний (точность 99,7 %), что делает его пригодным в роли метрики качества.
- Существующие подходы к автоматической формализации сопоставлены на едином наборе FormalMATH Lite [8] по единой методике оценивания, что показало

превосходство компактных специализированных моделей над крупными моделями общего назначения.

- Собран и очищен по семантической корректности обучающий корпус (около 30 тыс. пар) и обучена специализированная модель. Полученная модель превосходит одно из сильнейших существующих решений Kimina-Autoformalizer-7B [12], по ключевой семантической метрике (42,8 % против 39,3 % по расширенному Eq).
- Очищенный обучающий корпус и веса обученной модели опубликованы в открытом доступе.

2 Обзор литературы

2.1 Системы интерактивного доказательства теорем

Системы интерактивного доказательства теорем - это системы, в которых математические утверждения и их доказательства записываются на строго определённом формальном языке и механически проверяются по правилам логики. В отличие от обычных языков программирования, такие системы предназначены не для вычислений, а для кодирования и проверки логической корректности рассуждений: доказательство принимается лишь тогда, когда весь его текст полностью согласуется с правилами выбранной формальной системы.

В основе большинства таких систем лежит соответствие Карри–Ховарда: высказывания отождествляются с типами, а доказательства — с термами (программами) соответствующего типа. Базовым объектом такой системы выступает суждение вида

$$\Gamma \vdash a : A, \quad (1)$$

которое читается как «в контексте Γ терм a имеет тип A ». Если трактовать тип A как высказывание, то построение терма $a : A$ есть в точности доказательство A , а проверка доказательства сводится к проверке типов, то есть алгоритмически разрешимой процедуре. Тем самым логическая корректность рассуждения сводится к синтаксической проверке принадлежности терма типу, которую выполняет ядро системы. Всё остальное (тактики, автоматизация, удобный синтаксис) можно считать надстройкой, не входящей в число тех компонентов, которым нужно доверять. Различия между конкретными ассистентами во многом определяются тем, какая именно логика или теория типов положена в основу этого ядра.

Исторически одними из наиболее влиятельных стали Isabelle/HOL [1], построенный на логике высшего порядка, и Coq [2], опирающийся на исчисление индуктивных конструкций. Это разновидность теории типов, в которой типы могут зависеть от значений, а потому способны выражать произвольные математические высказывания в духе суждения (1). Различие оснований проявляется и в стиле работы: взаимодействие с такими системами обычно ведётся не написанием готового терма доказательства напрямую, а в режиме тактик, то есть команд, которые шаг за шагом преобразуют текущую цель, пока та не окажется доказанной, после чего система собирает из них проверяемый ядром терм. Обе системы обладают обширными библиотеками и на протяжении десятилетий применяются в формализации математики. На их фоне сравнительно недавно появилась система Lean, быстро ставшая одной из центральных площадок для формализации современной математики. Система Lean 4 [3] представляет собой одновременно систему интерактивного доказательства и полноценный язык программирования, основанный, как и Coq, на варианте исчисления индуктивных конструкций. Её отличительными чертами являются выразительная система типов, расширяемый метаязык, позволяющий определять собственные тактики и синтаксические конструкции прямо на самом Lean, и активно растущее сообщество. Не менее важную роль играет сопровождающая её библиотека mathlib [4]: крупная коллекция формализованной математики, охватывающая широкий спектр областей от алгебры до анализа и служащая стандартной целью для формализации новых результатов. Ключевое для дальнейшего изложения свойство Lean 4 состоит в том, что его компилятор выступает в роли надёжного оракула. Он проверяет не только синтаксическую правильность записи, но и логическую корректность доказательства, давая точную и автоматическую обратную связь о приемлемости формального текста. В основе этой проверки лежит уже упомянутая проверка типов, существенно опирающаяся на отношение равенства по определению, к которому мы вернёмся при обсуждении методов оценивания формализаций.

Несмотря на зрелость перечисленных инструментов, их практическое использование остаётся крайне трудоёмким даже для опытных специалистов. Каждый шаг неформального рассуждения, очевидный для человека, приходится развёртывать в несколько строк формального кода, а точность требуется на каждом логическом переходе. Это создаёт высокий барьер входа и существенно ограничивает масштабы, в которых математика может быть формализована вручную, затрудняя тем самым широкое распространение формальных методов.

С учётом описанных выше свойств автоматического контроля корректности доказательств ядром системы и богатой библиотеки `mathlib`, именно Lean 4 выступает в настоящей работе в качестве целевого формального языка.

2.2 Методы оценивания автоматической формализации

Автоматическая формализация - это задача перевода математических утверждений и доказательств, записанных на естественном языке, в формальный язык системы доказательства теорем. Её решение позволяет существенно снизить высокую стоимость ручной формализации и связать обширный корпус математических теорем, записанных лишь на естественном языке, с инструментами интерактивного доказательства теорем. При построении систем для решения данной задачи возникают следующие два основных вопроса: как построить такую систему/модель и как оценивать получаемую формализацию. Далее разберем подробнее вопрос оценки данной задачи.

Первые попытки оценивание автоматической формализации наследовали общепринятые методы из задачи машинного перевода. В классическом переводе качество измеряли поверхностными метриками совпадения с эталоном, например, при помощи одной из классических метрик BLEU, основанной на доле совпадающих n -грамм [13]. В ранних работах по автоматической формализации использовали именно эти метрики [14, 9]. Однако формальный язык системах интерактивного доказательства, в отличие от естественного, обладает строгой семантикой, и поверхностное сходство строк плохо согласуется с математической эквивалентностью. Одно и то же утверждение допускает множество синтаксически различных, но логически эквивалентных записей: переименование переменных, перестановку гипотез, β -редукцию (применение функции), развёртывание определений. Все эти преобразования резко снижают BLEU, хотя смысл утверждения не меняется и наоборот две почти совпадающие по строке формулировки могут оказаться семантически различными. Поэтому метрики n -граммного совпадения признаны для автоформализации ненадёжными.

Наличие компилятора-верификатора подсказывает иной, чисто символический критерий *проверки типов* (typecheck): долю формализаций, успешно проходящих компиляцию в выбранной системе, например, Lean 4. Этот критерий полезен, но недостаточен. Он отсеивает лишь синтаксически некорректные записи, пропуская формализации, которые компилируются, но не соответствуют исходному утверждению по смыслу. Иными словами, проверка типов является необходимым, но не достаточным условием правильной формализации.

Для примера возьмем тривиальное математическое утверждение на естественном языке:

$$\sin \frac{\pi}{4} = \cos \frac{\pi}{4} = \frac{1}{\sqrt{2}} \quad (2)$$

Рассмотрим подробнее различные формализации данного утверждения на языке Lean 4. На рисунке ниже (рисунок 1) приведена эталонная формализация и пример корректной формализации, полученной от модели. В формализации от модели значение $1/\sqrt{2}$ записано как $\sqrt{2}/2$, синус и косинус переставлены, а порядок конъюнктов

изменён, из-за чего при полной семантической эквивалентности BLEU = 0.27, что еще раз подчеркивает ненадежность такого подхода оценивания.

Эталонная формализация
<pre>theorem sin_cos_pi_div_four : Real.sin (π / 4) = Real.cos (π / 4) ∧ Real.sin (π / 4) = 1 / Real.sqrt 2 := by sorry</pre>
Формализация модели
<pre>theorem prediction : Real.cos (Real.pi / 4) = Real.sqrt 2 / 2 ∧ Real.sin (Real.pi / 4) = Real.sqrt 2 / 2 := by sorry</pre>

Рис. 1: Эталонная формализация утверждения на Lean 4 и эквивалентная ей формализация модели.

В следующем примере (рисунок 2) приведена формализация модели с явной ошибкой: вместо значения $1/\sqrt{2}$ записано $1/2$. При этом ни проверка типов, ни BLEU этой ошибки не выявляют (BLEU = 0.93).

Эталонная формализация
<pre>theorem sin_cos_pi_div_four : Real.sin (π / 4) = Real.cos (π / 4) ∧ Real.sin (π / 4) = 1 / Real.sqrt 2 := by sorry</pre>
Формализация модели
<pre>theorem prediction : Real.sin (π / 4) = Real.cos (π / 4) ∧ Real.sin (π / 4) = 1 / 2 := by sorry</pre>

Рис. 2: Эталонная формализация на Lean 4 и формализация модели, которая проходит проверку типов, но семантически неверна

Следующим шагом развития оценки автоматической формализации было использование втроенного в Lean 4 понятие *равенства по определению* (definitional equality) [3]. Это равенство, которое система интерактивного доказательства устанавливает сама, автоматически, путём вычислений. Неформально два выражения равны по определению, если одно можно привести к другому конечной последовательностью преобразований по определенным правилам. Именно набор таких правил и задаёт равенство по определению. Помимо сравнения термов с точностью до переименования связан-

ных переменных (α -эквивалентность), в документации Lean 4 выделены следующие, используемые правила конверсии [3]:

- β -редукцию - применение функции к аргументу (подстановку значения вместо связанной переменной)
- δ -редукцию - замену определённой константы её значением (развёртывание определения)
- ι -редукцию - вычисление рекурсора, применённого к конструктору (примитивную рекурсию)
- ζ -редукцию - подстановку значений `let`-связанных переменных

Терм, в котором проведены все возможные редукции, называется *нормальной формой*. Таким образом, проверка равенства по определению сводится к приведению обоих выражений к нормальной форме и их сравнению. Принципиально важно, что это отношение проверяется ядром Lean 4 автоматически. Тем самым в самой системе уже заложен полностью автоматический инструмент проверки эквивалентности двух формализаций одного утверждения. Это как раз то, чего лишены и BLEU, и проверка типов. На рисунке 3 приведен пример, в котором формализация модели совпадает по смыслу с эталонной формализацией, но при этом за счет переобозначения $\alpha = \frac{\pi}{4}$ классические метрики будут низкими. Однако за счет применения ζ -редукции равенство по определению сработает и такая формализация будет считаться корректной.

Эталонная формализация
<pre>theorem sin_cos_pi_div_four : Real.sin (π / 4) = Real.cos (π / 4) ∧ Real.sin (π / 4) = 1 / Real.sqrt 2 := by sorry</pre>
Формализация модели
<pre>theorem prediction : let a := π / 4 Real.sin a = Real.cos a ∧ Real.sin a = 1 / Real.sqrt 2 := by sorry</pre>

Рис. 3: Пример корректной формализации. Применение ζ -редукции сводит к эталонной.

В чистом виде равенство по определению оказывается слишком строгим. Оно жёстко завязано на определение объектов и фиксированный набор правил конверсии, поэтому многие интуитивно очевидные равенства им не схватываются. Классический пример: для натурального n выражения $n+0$ и n равны по определению, тогда как $0+n$ и n уже нет, поскольку сложение на `Nat` определено рекурсией по второму аргументу и на переменном n вычисление «застревает». В итоге, прямое использование равенства по определению как критерия эквивалентности формализаций имеет определенные ограничения и сильно занижает получаемые метрики.

Именно эту чрезмерную строгость снимает предложенная в работе [11] метрика BEq (Bidirectional Extended Definitional Equivalence). Идея состоит в том, чтобы расширить равенство по определению, допустив ограниченный набор преобразований.

Формализуется это следующим образом. Обозначим равенство по определению через $\sim_D$. Пусть R - множество допустимых преобразований (на практике небольшой набор тактик Lean), а $U(s, R)$ - множество всех корректных утверждений, получаемых применением преобразований из R к утверждению s . Введем функцию преобразования $T(s_P \mid s_Q, R)$, которая пытается переписать s_P так, чтобы получить нечто равное по определению s_Q , используя преобразования из R :

$$T(s_P \mid s_Q, R) = \begin{cases} s'_P, & \text{если } \exists s'_P \in U(s_P, R) : s_Q \sim_D s'_P, \\ \perp, & \text{если } \forall s'_P \in U(s_P, R) : s_Q \not\sim_D s'_P, \end{cases} \quad (3)$$

где $\perp$ — фиктивное утверждение, не равное по определению никакому корректному (сигнал «преобразование не найдено»). Проще говоря, T берёт s_P и, пользуясь только разрешёнными преобразованиями, старается «подогнать» его под s_Q , если это удаётся, возвращается подогнанный вариант, иначе отказ.

Чтобы такой поиск можно было вести на практике, утверждение переводится в цель доказательства: в режиме тактик утверждение s_P представляется целью, а тактики переписывают цель, не меняя её смысла (преобразования происходят на синтаксическом уровне, семантика сохраняется). Тогда R конкретизируется как набор тактик Lean, и вычисление T сводится к поиску короткой последовательности тактик, превращающей одно утверждение в другое.

Определение T является сложной задачей, так как количество тактик и способов переписать исходное выражение слишком велико. Её конкретный выбор — это вопрос реализации, и в литературе встречаются разные варианты. Сам вопрос «существует ли подходящая последовательность тактик» неконструктивен, а пространство «тактика вместе с её аргументами» (какую лемму применить, что ввести через **have**, чем переписать через **rw**) слишком велико для слепого перебора, поэтому требуется та или иная эвристическая процедура поиска. Её конкретный выбор это вопрос реализации и от нее будет зависеть качество оценки. В исходной работе [11] поиск ведёт языковая модель, обученная на формальной математике. Она предлагает правдоподобные цепочки тактик. Другой вариант это детерминированный символьный поиск по небольшому заранее заданному фиксированному набору правил. Такой вариант работает не требует большой языковой модели, воспроизводим и легко масштабируется на множество кандидатов.

При любом способе поиска сохраняется ключевое свойство, обеспечивающее надёжность метрики: процедура лишь предлагает цепочку тактик, а итоговую корректность устанавливает ядро Lean, символически её проверяя. Неверную цепочку ядро отвергает, поэтому выбор поисковой эвристики (языковая модель или набор правил) влияет лишь на полноту и скорость, но не на корректность: ошибки приводят только к ложноотрицательным результатам (эквивалентность не подтверждена), но не к ложноположительным (ничего лишнего не объявляется эквивалентным). На выборке из 200 пар утверждений с экспертной разметкой BEq повысила точность определения эквивалентности с 82,50% до 90,50%, а аккуратность (precision) — с 70,59% до 100,0%.

Помимо метрик, область опирается на ряд стандартных бенчмарков и наборов данных. Набор MINIF2F [6] содержит 488 утверждений задач олимпиадного уровня (AIME, AMC, IMO) и задуман как единый кросс-системный эталон, представленный сразу в нескольких языках (Lean, Isabelle, Metamath, HOL Light). ProofNet [7] включает 371 задачу университетского уровня по анализу, линейной и абстрактной алгебре

и топологии, причём каждая снабжена формальным и неформальным утверждениями и неформальным доказательством, что делает его удобным именно для оценки автоформализации. Более масштабный FormalMATH [8] насчитывает 5560 формально проверенных утверждений на Lean 4 — от школьных олимпиад до университетского уровня; характерно, что даже сильнейшие современные пруверы доказывают на нём лишь около 16,5% задач, что подчёркивает разрыв между формализацией и доказательством. Для обучения моделей используются крупные синтетические корпуса информально-формальных пар, такие как Lean Workbook [15] (≈ 57 тыс. задач на Lean 4) и FORML4 [16].

2.3 Подходы к решению задачи автоматической формализации

Ранние нейросетевые подходы. В первых попытках применить нейронные сети к формализации математики задача рассматривалась как машинный перевод и решалась теми же архитектурами, что и классический машинный перевод между естественными языками: рекуррентным кодировщиком–декодировщиком с механизмом внимания. Сам принцип работы устроен следующим образом: последовательность токенов на исходном языке подаётся на вход кодировщика, после чего декодировщик авторегрессионно порождает последовательность на целевом языке.

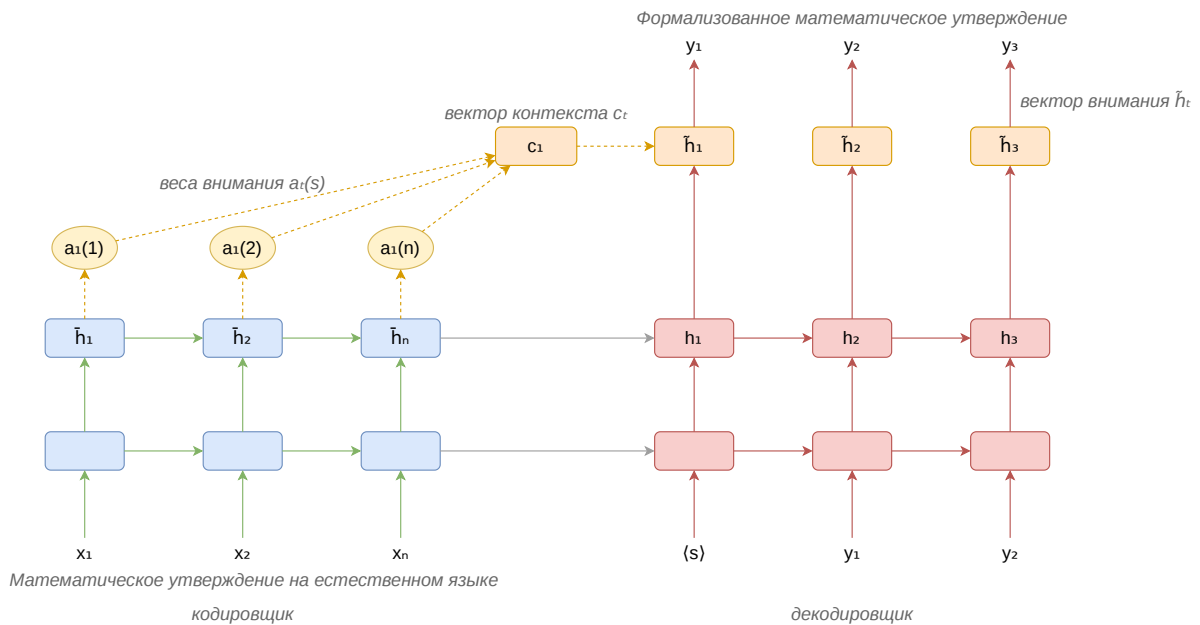


Рис. 4: Принцип работы формализации математических теорем при помощи рекуррентной модели с механизмом внимания

Формально, следуя постановке задачи нейронного машинного перевода [17], пусть $x = (x_1, \dots, x_n)$ — последовательность токенов неформального утверждения, а $y = (y_1, \dots, y_m)$ — токены его формализации над словарём формального языка. Тогда модель с параметрами θ напрямую задаёт условную вероятность $p_\theta(y | x)$ того, что формализацией неформального утверждения x служит последовательность y :

$$p_\theta(y | x) = \prod_{t=1}^m p_\theta(y_t | y_{<t}, x), \quad y_{<t} := (y_1, \dots, y_{t-1}). \quad (4)$$

Кодировщик отображает x в набор скрытых состояний $\bar{h}_1, \dots, \bar{h}_n$. На шаге декодирования t механизм внимания сравнивает текущее состояние декодировщика h_t с состояни-

ями кодировщика, получая веса внимания a_t и вектор контекста c_t :

$$a_t(s) = \frac{\exp(\text{score}(h_t, \bar{h}_s))}{\sum_{s'=1}^n \exp(\text{score}(h_t, \bar{h}_{s'}))}, \quad c_t = \sum_{s=1}^n a_t(s) \bar{h}_s. \quad (5)$$

Вектор контекста объединяется со скрытым состоянием декодировщика в вектор внимания $\tilde{h}_t$, по которому предсказывается очередной токен:

$$\tilde{h}_t = \tanh(W_c [c_t; h_t]), \quad p_\theta(y_t | y_{<t}, x) = \text{softmax}(W_s \tilde{h}_t). \quad (6)$$

Величины a_t , c_t и $\tilde{h}_t$ на рис. 4 отвечают весам внимания, вектору контекста и вектору внимания соответственно. Обучение по параллельному корпусу $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ выровненных пар сводится к минимизации суммарного отрицательного логарифмического правдоподобия

$$J(\theta) = \sum_{(x,y) \in \mathcal{D}} (-\log p_\theta(y | x)), \quad (7)$$

минимизируемой по параметрам θ , а на этапе вывода ищется наиболее вероятная формализация $\hat{y} = \arg \max_y p_\theta(y | x)$, что на практике приближается жадным или лучевым поиском.

Обучение такой модели требует параллельного корпуса выровненных пар предложений, что и определило центральную трудность раннего этапа. Существенно, что в отличие от более поздних работ, эти модели обучались с нуля непосредственно на задаче перевода. Авторы работы [14] обучали такую модель переводить записанные в L^AT_EX тексты на языке Mizar в формальные утверждения Mizar. В последующей работе [18] были сопоставлены три модели нейронного машинного перевода на четырёх подготовленных наборах данных. Помимо обучаемой с нуля модели с учителем, авторы рассматривали модели обучения без учителя. Тем самым идея предобучения возникает в области уже здесь, однако предобучение проводится на собственных доменных корпусах, а не переносится из крупных моделей общего назначения. Эти работы зафиксировали два сквозных для области мотива: дефицит выровненных пар и использование семантических средств (типизации, проверки типов) для контроля качества формализаций.

Применение больших языковых моделей. Появление больших языковых моделей (БЯМ) сместило фокус с обучаемых с нуля моделей перевода на использование предобученных моделей. Современные БЯМ это нейросети на основе архитектуры трансформера [5], ключевым элементом которой является механизм внимания (ср. ур. (5)), применяемый в том числе внутри самой последовательности (self-attention). Это позволяет распараллеливать вычисления и обучать модели с миллиардами параметров. Хотя исходный трансформер был предложен как пара кодировщик–декодировщик, большинство современных БЯМ представляют собой модели «только декодировщик» (decoder-only). Неформальное утверждение x подаётся как часть запроса (контекста), а формализация y достраивается как продолжение последовательности, что напрямую соответствует факторизации (4).

В работе [9] было показано, что предобученные большие языковые модели (Codex и PaLM) обладают заметной способностью к формализации, активируемой контекстным обучением (подробнее в главе 2.4) без дообучения. На выборке задач датасета MATH модель Codex безошибочно переводила в спецификации Isabelle/HOL 25,3% задач, причём большинство ошибок объяснялось рассогласованием неформальных и формальных определений, а качество росло с увеличением размера модели. Авторы также

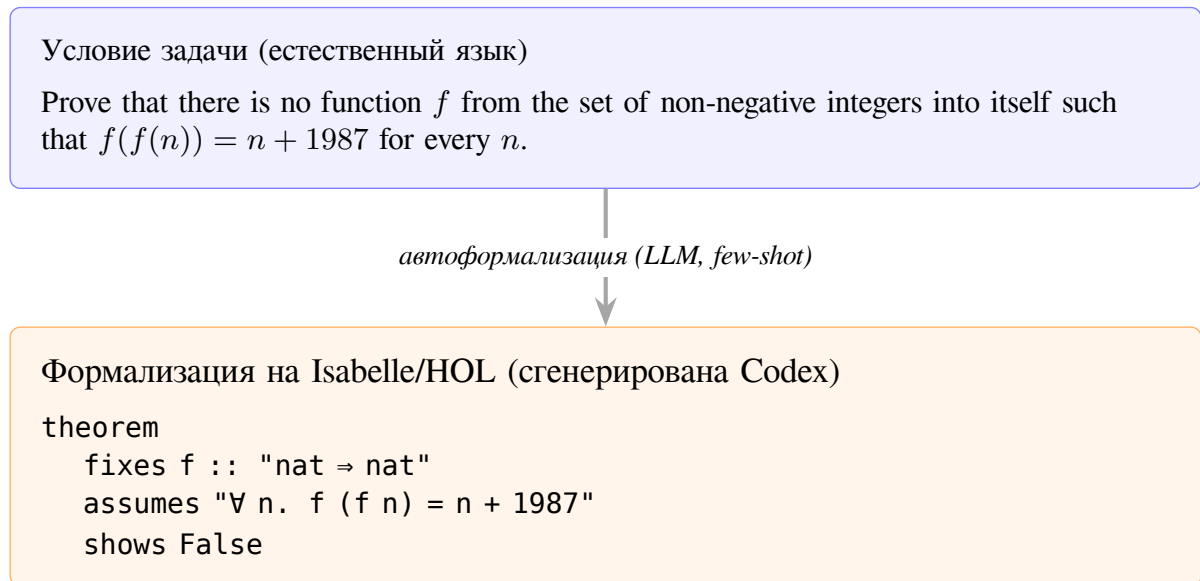


Рис. 5: Пример автоматической формализации математической теоремы [9]

продемонстрировали практическую пользу автоматической формализации: для задачи доказательства теорем дообучение модели на автоматически формализованных теоремах улучшило долю доказанных задач на MINIF2F с 29,6% до 35,2%.

2.4 Адаптация больших языковых моделей к задачи автоматической формализации

Контекстное обучение. Простейшим способом адаптации предобученной большой языковой модели к задаче определенного домена является контекстное обучение (In-context learning). В случае задачи автоматической формализации в этом методе несколько примеров «неформальное утверждение - его формализация» подаются прямо в промпт, а веса модели при этом никак не изменяются [19]. Как отмечалось выше, в работе [9] показали, что таким образом активизируется заметная способность Codex и PaLM к переводу в Isabelle/HOL. Систематические замеры на ProofNet подтвердили ту же картину для Lean: при 12 примерах формализации модель Codex давала 13,4% корректных формализаций при доле проходящих типизацию записей 23,7% [7]. Извлечение релевантных примеров, при котором в промпт подбираются не одни и те же примеры, а наиболее близкие к целевому утверждению пары, поднимало метрики до 16,1% и 45,2% соответственно.

Еще одним важным приемом, который позволяет повысить качество в сложных задачах, является цепочка рассуждений (chain-of-thought) [20]. В этом методе модель в начале приводит промежуточные шаги рассуждения и лишь затем составляет итоговый ответ. На неформальных математических задачах рассуждение по цепочке заметно повышает качество, но при этом требует большего количества вычисления для получения финального ответа.

Несмотря на то, что вышеописанные методы позволяют увеличить качество автоматической формализации при помощи БЯМ, метод контекстного обучения является достаточно ограниченным. Он крайне чувствителен к подбору примеров и формулировке запроса, не изменяет весов модели и поэтому плохо усваивает синтаксис Lean 4. На задаче формализации это выражается в стабильно низких долях типизируемых и тем более семантически верных записей. Именно эти ограничения мотивируют переход

к дообучению с учителем.

Обучение с учителем. Дообучение с учителем применяют к уже предобученной модели, при этом сам процесс обучения остается таким же, что и раннее при предобучении модели (7). В случае автоматической формализации дообучение происходит на относительно небольшом корпусе пар «неформальное утверждение - формализация». В отличие от контекстного обучения, дообучение с учителем изменяет веса модели и поэтому такой подход способен надёжно привить ей синтаксис и стиль формального языка Lean 4. В работе [10] автоматическая формализация и последующая фильтрация использовались для создания около 8 миллионов синтетических пар «утверждение—доказательство» на Lean 4, после чего модель DeepSeekMath 7B, дообученная на этом корпусе, достигла точности 46,3% на MINIF2F, превзойдя GPT-4 (23,0%).

Применительно к задаче автоматической формализации центральным препятствием для обучения с учителем остаётся отсутствие крупного корпуса пар. Ключевое наблюдение, состоит в том, что информализация даётся языковым моделям значительно легче, чем формализация: на продвинутой математике авторы работы [9] получили около 76% приемлемых информализаций против 25% корректных прямых формализаций. Это превращает информализацию в дешёвый источник обучающих пар.

На этой идее построен ряд корпусов. Набор данных из работы [21] получен информализацией библиотек mathlib (Lean 4) и Archive of Formal Proofs (Isabelle) при помощи GPT-4 и содержит около 332 тысяч пар. Lean Workbook [15] формализует задачи олимпиадных форумов в Lean 4 при помощи многоступенчатой фильтрации (компилятор, проверка на логическое следование, ручная диагностика), наращивая корпус примерно до 57 тысяч задач (из них около 5 тысяч с формальными доказательствами) при заявленной точности случайной выборки 93,5%. Набор FORML4 [16] получен информализацией mathlib моделью Gemini-Pro-1.5 и, в отличие от большинства аналогов, содержит не только утверждения, но и формальные доказательства. Наконец, корпус FormalMATH собирают автоматически: формализацию параллельно ведут несколько языковых моделей, после чего результат проходит автоматическую верификацию и финальную проверку экспертами. Через все ступени отбора проходит лишь 21,7% синтаксически и семантически корректных утверждений.

Накопленные корпуса позволили перейти к дообучению больших языковых моделей непосредственно для задачи автоматической формализации. Обучение на корпусе из работы [21] подняло долю приемлемых формализаций модели LLaMA с 0% у базовой версии до 16–18% у дообученной, причём многоязычное обучение (Lean и Isabelle совместно) улучшало качество даже на одноязычных задачах и оказывалось эффективнее одноязычного. Доля проходящих компиляцию записей на Lean выросла при этом до 20% на MINIF2F. В работе [15] был представлен подход с итеративным обучением с промежуточной проверкой экспертами. После обучения модели на первой итерации, эксперты диагностировали ошибки и затем исправленные пары помещались в выборку для обучения. В работе [16] был предложен подход Process-Driven Autoformalization: к процессу обучения с учителем добавляется точная обратная связь компилятора Lean 4. Выяснилось, что оценка отдельных шагов превосходит исходную оценку лишь итогового результата по всем рассматриваемым показателям. Одной из актуальных работ является [22], в ней авторы дообучали базовую модель Qwen2.5-Coder-7B на парах соревновательного уровня, после чего итеративно улучшали качество модели при помощи оценки модели судьи.

Сопоставление этих работ затрудняется разнородностью метрик. В основном для оценки использовали метрики: доля проходящих проверку типов записей и BLEU. При этом как отмечалось ранее данные метрики являются достаточно шумными. Более точ-

ные методы оценки эквивалентности эталонной формализации, такие как BEq и её различные расширения появились совсем недавно.

Рассуждающие языковые модели Некоторые современные большие языковые модели обладают выраженной способностью к *рассуждению*: генерации промежуточных шагов вывода перед итоговым ответом. Такое поведение в первую очередь прививается на стадии обучения с учителем. Модель дообучают на примерах, которые уже содержат примеры рассуждений, и тем самым задают и формат, и саму привычку «думать перед ответом». На следующей стадии модель обучают при помощи методов обучения с подкреплением и тем самым усиливает и оттачивает уже заложенное поведение [23]. Подробнее об использовании методичекого обучения с подкреплением будет рассказано в следующем параграфе. Показательно, что вариант без такого с использованием только обучения с подкреплением также порождает рассуждения, но они выходят плохо читаемыми и со смешением языков. Именно поэтому демонстрации рассуждений на стадии обучения с учителем используют как отправную точку.

Тот же подход был перенесён на формальную математику, а именно к задаче доказательства теорем. В работе [22] авторы ввели структурированный «формальный паттерн рассуждения»: внутри блока `<think>` модель проводит промежуточные размышления и преобразования на естественном языке, а затем собирает из них итоговое доказательство на формальном языке Lean 4 (рисунок 6). Чтобы заложить это поведение, в работе было использовано около 20 тысяч синтетических примеров, в которых блок рассуждения соединяет неформальное и формальное доказательства, и лишь затем переходят к обучению с подкреплением. В результате наблюдается рост длины рассуждения с трудностью задачи и улучшение качества с увеличением размера модели, ранее для нейросетевых пруверов нехарактерное. Такой подход и позволил достичь рекордных 80,7% на MINIF2F.

Вход (запрос)

```
-- 7-й член прогрессии = 30, 11-й = 60. Найти 21-й.
theorem arith (a d : ℝ) (h₀ : a + 6 * d = 30)
  (h₁ : a + 10 * d = 60) :
  a + 20 * d = 135 := by
```

Выход (ответ модели)

```
<think>
Вычитая  $h_0$  из  $h_1$ :  $4d = 30$ , значит  $d = 15/2$  и  $a = -15$ .
Тогда  $a + 20d = 135$ . Запишем доказательство тактиками:
</think>
```

```
have hd : d = 15 / 2 := by linarith
have ha : a = -15    := by linarith
linarith
```

Рис. 6: Шаблон «рассуждения перед формализацией» в формате «запрос – ответ» (по образцу формального паттерна рассуждения из работы [22]). Пример приведён для тривиальной задачи об арифметической прогрессии.

Для задачи автоматической формализации этот приём до сих пор практически не использовался: модели, как правило, порождают формализацию напрямую из неформального утверждения, без явного промежуточного рассуждения. Между тем именно

здесь такой шаг выглядел бы особенно уместным, поскольку, основная масса ошибок в формализации объясняется не синтаксисом, а рассогласованием неформальных и формальных определений и неявными предположениями. Однако обучить модель для данной задачи таким образом, возможно лишь при наличии обучающих примеров с рассуждениями для самой задачи формализации. Существующие корпуса состоят из пар «утверждение – формализация» без каких-либо следов рассуждения, поэтому перенос подхода с рассуждениями на автоматическую формализацию упирается не столько в архитектуру, сколько в отсутствие подходящих данных, что и делает это направление естественным, но всё ещё малоисследованным.

Обучение с подкреплением. Параллельно с дообучением с учителем в задаче доказательства теорем закрепилось обучение с подкреплением. Формально общая постановка задачи обучения с подкреплением задается в виде марковского процесса принятия решений:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma), \quad (8)$$

где $\mathcal{S}$ - множество состояний, $\mathcal{A}$ - множество действий, $P(s' | s, a)$ - вероятность перехода в следующее состояние, $r(s, a)$ — награда, а $\gamma \in [0, 1]$ — коэффициент дисконтирования. Политика $\pi_\theta(a | s)$ выбирает действие в текущем состоянии. Обучение состоит в максимизации ожидаемой суммарной награды:

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right], \quad (9)$$

где $\tau = (s_0, a_0, s_1, a_1, \dots)$ — траектория, порождённая политикой и динамикой среды.

Для больших языковых моделей эта абстрактная постановка почти напрямую превращается в задачу генерации текста. Чтобы не вводить новые обозначения, сохраним x и y из постановки автоматической формализации выше: x обозначает входной запрос, а $y = (y_1, \dots, y_m)$ - порождаемую последовательность токенов. Для фиксированного x задаётся $\mathcal{M}_x$ - состояние на шаге t есть уже сгенерированный префикс $s_t = (x, y_{<t})$, действие a_t - выбор следующего токена y_t , а переход детерминированно дописывает этот токен к префиксу. Эпизод завершается при генерации специального токена конца последовательности или при достижении максимальной длины. Тогда авторегрессионное распределение из (4) можно рассматривать как политику

$$\pi_\theta(y | x) = \prod_{t=1}^m \pi_\theta(y_t | x, y_{<t}) \equiv p_\theta(y | x), \quad (10)$$

а награда обычно задаётся только в конце эпизода как $r(x, y)$ и оценивает весь сгенерированный ответ целиком.

Из этой схемы получаются две близкие, но принципиально разные задачи. В доказательстве теорем x - это формальное утверждение вместе с контекстом библиотеки, y - полученное от модели доказательство на Lean, а награда может быть бинарной: $r(x, y) = 1$, если Lean принимает доказательство, и $r(x, y) = 0$ иначе. В автоматической формализации x - неформальное математическое утверждение, y - его формализация на Lean, а награда должна измерять не только компилируемость, но и смысловое соответствие исходному утверждению. Именно это различие оказывается ключевым: для доказательства награда верифицируема компилятором, тогда как для автоформализации её трудно определить автоматически.

На практике цель (9) для больших языковых моделей дополняют ограничением на отклонение от исходной дообученной с учителем модели π_{ref} , чтобы модель не

разрушала уже выученный стиль генерации. Поэтому часто оптимизируют регуляризованную цель вида

$$\max_{\theta} \mathbb{E}_{x \sim \rho, y \sim \pi_{\theta}(\cdot | x)} [r(x, y) - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))], \quad (11)$$

где ρ - распределение входных запросов, D_{KL} - дивергенции Кульбака — Лейблера (KL-регуляризация) удерживает распределение ответов близким к опорной модели, а коэффициент β задаёт силу этого ограничения.

Одним из наиболее используемых вариантов такой оптимизации стал алгоритм GRPO (Group Relative Policy Optimization) [24]. Он родственен PPO, но устраняет необходимость обучать отдельную модель ценности. Для каждого запроса x текущая политика порождает группу из G ответов $\{y_i\}_{i=1}^G$, для каждого ответа вычисляется награда $r_i = r(x, y_i)$, а затем награды нормируются внутри группы. Типичная оценка преимущества имеет вид

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)}, \quad (12)$$

то есть ответ получает положительное преимущество, если он лучше других ответов той же модели на тот же запрос. Далее параметры обновляются так, чтобы повышать вероятность токенов в ответах с положительным $\hat{A}_i$ и понижать вероятность ответов с отрицательным $\hat{A}_i$, одновременно штрафую отклонение от опорной политики. Упрощённо GRPO можно понимать как обучение не на абсолютных оценках качества, а на относительном ранжировании нескольких примеров внутри одной группы.

Ключевая причина, по которой обучение с подкреплением естественно ложится на доказательство теорем - это проверяемость награды. Компилятор Lean однозначно принимает или отвергает доказательство, давая надёжный бинарный сигнал. В таком случае функция $r(x, y)$ не требует ручной разметки и не зависит от субъективного судьи. На этом принципе построен модель из работы [22] — обучения с подкреплением поверх базовой модели Qwen2.5-72B, формирующий структурированный «формальный» стиль рассуждения и устанавливающий новый рекорд на MINIF2F (80,7%). Кроме этого подобный подход был применен в работе DeepSeek-Prover [10].

Для задачи автоматической формализации обучение с подкреплением, и в частности GRPO, до сих пор практически не применялось. По-прежнему доминируют дообучение с учителем и экспертные итерации. Причина кроется в устройстве награды. В отличие от доказательства, где успех удостоверяется компилятором, при автоформализации компилятор проверяет лишь типовую корректность записи, но не её смысловое соответствие исходному утверждению. Как отмечалось в разделе 2.3, проверка типов необходима, но не достаточна. Надёжного, дешёвого и автоматического сигнала вознаграждения за правильную по смыслу формализацию не существует, а оценщики на основе больших языковых моделей недетерминированы и склонны к ложноположительным срабатываниям. Авторы Kimina-Prover прямо отмечают, что сигнал награды в данной задаче размыт, а обратная связь от модели-судьи ненадёжна, и называют применение обучения с подкреплением в невалидируемых доменах серьёзным открытым вызовом [22]. Именно поэтому разработка достоверных автоматических метрик эквивалентности, таких как BEq [11], рассматривается как необходимый шаг к построению верифицируемой награды и, в перспективе, к переносу подходов обучения с подкреплением на задачу автоматической формализации.

2.5 Ограничения существующих подходов.

Проведённый обзор позволяет выделить несколько системных пробелов, общих для существующих работ по автоматической формализации.

Во-первых, область до сих пор лишена единого метода оценивания: разные работы измеряют качество разными и зачастую несопоставимыми способами — от поверхностной метрики BLEU и доли проходящих проверку типов записей до равенства по определению. Каждый из этих критериев, как было показано выше, либо ненадёжен, либо чрезмерно строг. Надёжные метрики семантической эквивалентности эталонной формализации, такие как BEq [11], появились лишь недавно и пока не стали общепринятым стандартом. Из-за этого результаты разных работ напрямую несопоставимы, заявленные показатели плохо переносятся между статьями, а сам прогресс в задаче трудно измерить единообразно.

Во-вторых, приём явного рассуждения перед ответом, заметно повышающий качество как в доказательстве теорем, так и на неформальных математических задачах, для автоматической формализации фактически не использовался: модели порождают формализацию напрямую из неформального утверждения, без промежуточного рассуждения, согласующего неформальные и формальные определения. Причина заключается в том, что существующие корпуса состоят из пар «утверждение – формализация» без каких-либо следов рассуждения именно для задачи формализации.

В-третьих, методы, доказавшие свою эффективность в смежной задаче доказательства теорем, для автоматической формализации остаются практически неосвоенными. В частности, обучение с подкреплением и, в особенности, алгоритм GRPO [24], ставший стандартом при обучении рассуждающих моделей и моделей для доказательства теорем. Главным препятствием выступает отсутствие проверяемого сигнала награды. Компилятор удостоверяет лишь типовую корректность, но не смысловое соответствие, и именно надёжная автоматическая метрика эквивалентности оказывается недостающим звеном для переноса этих подходов.

Совокупность этих пробелов: разнородность и ненадёжность оценивания, отсутствие рассуждений в задаче автоматической и неиспользование обучения с подкреплением и, в частности, GRPO определяет направление настоящей работы.

3 Методы оценивания автоматической формализации

3.1 Постановка задачи

Как было показано в обзоре литературы (раздел 2.3), центральным препятствием для развития автоматической формализации остаётся не столько построение моделей, сколько отсутствие надёжного и автоматического сигнала о семантической правильности формализации. Проверка типов необходима, но не достаточна, а метрики n -граммного совпадения вроде BLEU ненадёжны. Экспертная оценка точна, но дорога и не масштабируется. Данная глава посвящена систематическому сравнению автоматических методов оценивания и обоснованию того, какой из них пригоден в качестве метрики качества и в качестве награды при обучении с подкреплением.

Формализуем задачу. Обозначим через $\mathcal{X}$ множество математических утверждений на естественном языке, а через $\mathcal{F}$ множество синтаксически допустимых формализаций на языке Lean 4. Каждому утверждению $x \in \mathcal{X}$ сопоставлена эталонная формализация $y^* \in \mathcal{F}$, написанная экспертами и считающаяся корректной. Пусть $\hat{y} \in \mathcal{F}$ - проверяемая формализация-кандидат, порождённая моделью автоматической формализации. Кандидат $\hat{y}$ называется *корректным*, если он семантически эквивалентен смыслу утверждения x , то есть эталону y^* , а не просто синтаксически на него похож. Истинную экспертную метку корректности обозначим

$$g(x, y^*, \hat{y}) \in \{0, 1\}, \quad (13)$$

где $g = 1$ соответствует семантически корректной формализации, а $g = 0$ — некорректной.

Тогда методом автоматического оценивания называется отображение

$$m: \mathcal{X} \times \mathcal{F} \times \mathcal{F} \rightarrow \{0, 1\}, \quad (x, y^*, \hat{y}) \mapsto m(x, y^*, \hat{y}), \quad (14)$$

выносящее автоматический вердикт о корректности кандидата $\hat{y}$ без участия эксперта. Рассматриваемые далее методы используют разные аргументы: проверка типов опирается лишь на кандидата $\hat{y}$, символические методы семейства BEq сравнивают кандидат $\hat{y}$ с эталоном y^* , а судья на основе большой языковой модели использует все три аргумента, включая утверждение на естественном языке x .

Цель состоит в выборе метода m , наиболее точно приближающего экспертную метку g . Поскольку и m , и g принимают значения в $\{0, 1\}$, качество метода естественно измерять, используя классические метрики для оценивания из задачи бинарной классификации:

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}, \quad (15)$$

где TP , FP и FN - число истинно-положительных, ложноположительных и ложноотрицательных вердиктов метода относительно метки g . Точность (precision) - это доля действительно корректных формализаций среди признанных корректными, полнота (recall) - доля найденных среди всех корректных, F_1 - их гармоническое среднее, а доля правильных ответов (ассигасу) - доля совпадений вердикта метода с экспертной меткой на всей выборке.

Принципиальная трудность состоит в том, что сами методы оценивания нужно с чем-то сравнивать. В разделе 3.2 описывается размеченный экспертами набор данных, в котором для каждой формализации заранее известна метка g . Он играет роль «золотого стандарта» и позволяет измерять качество самих методов оценивания. Затем

на этом наборе последовательно проверяются символьные методы на основе компилятора Lean 4 (раздел 3.3) и методы на основе больших языковых моделей (раздел 3.4), после чего проводится итоговое сравнение по качеству, вычислительной стоимости и верифицируемости (раздел 3.5).

3.2 Данные для оценивания

Чтобы измерить качество метода оценивания, недостаточно набора пар «утверждение – формализация». Требуется выборка формализаций, для каждой из которых *экспертно* установлено, соответствует ли она исходному утверждению по смыслу. Только располагая такой эталонной разметкой, можно трактовать любой автоматический метод как бинарный классификатор («корректна / некорректна») и измерять его аккуратность, полноту и долю ложных срабатываний.

За основу взят набор ProofNet [7], в нем содержится 371 задача университетского уровня по математическому анализу, линейной и абстрактной алгебре и топологии, где каждое утверждение снабжено неформальной и эталонной формальной записью. Поверх него строится набор данных для оценки методов следующим образом. Для каждого утверждения / теоремы собирается несколько кандидатов формализаций: как эталонные, так и порождённые различными моделями, включая намеренно ошибочные варианты (искажение констант, перестановка кванторов, подмена гипотез, развёртывание определений). Каждый кандидат размечается экспертом меткой: 0 - не корректная формализация, 1 - корректная формализация, фиксирующей именно семантическую эквивалентность эталону, а не синтаксическое сходство. Далее полученные метки используются как истинная оценка и проводится сравнение оценок автоматических методов с истинной.

В рамках данной работы исходный набор данных был расширен. Во-первых, добавлены дополнительные кандидаты к утверждениям ProofNet, чтобы плотнее покрыть типичные ошибки моделей и пограничные случаи (формализации, проходящие типизацию, но семантически неверные, и, наоборот, синтаксически непохожие, но эквивалентные эталону). Во-вторых, для расширения предметной области в выборку включены размеченные примеры из набора CombiBench [25] - комбинаторные и олимпиадные задачи, стилистически и тематически далёкие от «библиотечных» утверждений `mathlib`. Это важно, поскольку, как отмечалось в разделе 2.3, расхождение стиля `mathlib` и соревновательных задач — известный источник смещения, и метод оценивания должен сохранять надёжность на обоих типах данных.

Итоговый набор содержит 245 уникальных неформальных утверждений и 1464 формализации-кандидата (в среднем около 6 кандидатов на утверждение, максимум 17). Из них корректными являются 486 формализаций (33,2%) и некорректными 978 (66,8%). Каждая запись содержит поля: идентификатор задачи, неформальное утверждение, заголовок файла Lean 4 (используемые дополнительные библиотеки и открытые пространства имён), эталонную формализацию, формализацию-кандидат и экспертную метку. Пример записи приведён на рисунке 7.

Дисбаланс классов (1 : 2 в пользу некорректных) и наличие большого числа компилируемых, но семантически неверных кандидатов делают этот бенчмарк подходящим для проверки: метод, который просто проверяет компилируемость, на нём заведомо проявит свою недостаточность, тогда как метод, действительно улавливающий семантическую эквивалентность, должен одновременно сохранять высокую полноту на корректных кандидатах и низкую долю ложных срабатываний на некорректных. В дальнейшем все метрики методов оценивания вычисляются именно относительно экспертной метки на этой выборке из 1464 примеров.

УТВЕРЖДЕНИЕ НА ЕСТЕСТВЕННОМ ЯЗЫКЕ Докажите, что многочлен $x^3 + 3x + 2$ неприводим над полем рациональных чисел $\mathbb{Q}$.
ЗАГОЛОВОК ФАЙЛА <pre>import Mathlib open Fintype Set Real Ideal Polynomial open scoped BigOperators</pre>
ЭТАЛОННАЯ ФОРМАЛИЗАЦИЯ <pre>theorem exercise_4_6_2 : Irreducible (X^3 + 3*X + 2 : Polynomial ℚ) :=</pre>
ФОРМАЛИЗАЦИЯ-КАНДИДАТ <pre>theorem dummy (f : Polynomial ℚ) (hf : f = X^3 + 3*X + 2) : Irreducible f := sorry</pre>
ЭКСПЕРТНАЯ МЕТКА корректна (= 1)

Рис. 7: Пример записи из бенчмарка для оценки методов оценивания. Для неформального утверждения заданы заголовок файла, эталонная формализация и формализация-кандидат

3.3 Методы автоматической оценки на основе компилятора Lean 4

Первый тип методов опирается исключительно на компилятор Lean 4 и не использует внешних моделей. Его привлекательность в том, что итоговое решение об эквивалентности выносит ядро Lean, то есть метод является полностью интерпретируемым и проверяемым: он не может «придумать» эквивалентность, которой нет.

Проверка типов. Простейший критерий, подробно разобранный в разделе 2.3, доля формализаций, успешно проходящих компиляцию. Формализация признаётся корректным тогда и только тогда, когда он типизируется в Lean 4 с заданным заголовком файла. Этот критерий отсеивает синтаксически некорректные записи, но не способен отличить компилируемую, но семантически неверную формализацию от верной. Он является необходимым, но не достаточным условием правильности.

Расширенное равенство по определению (BEq). Содержательный символический критерий даёт метрика BEq [11], описанная в разделе 2.3. Кандидат s_P признаётся эквивалентным эталону s_Q , если найдётся последовательность семантически сохраняющих преобразований из допустимого набора R , приводящая s_P к выражению, равному s_Q по определению (см. формулу (3)). В базовой реализации BEq поиск ведётся детерминированным символьным перебором по небольшому фиксированному набору тактик,

а итоговую корректность во всех случаях устанавливает ядро Lean. Это обеспечивает ключевое свойство: ошибки поиска порождают лишь ложноотрицательные результаты (эквивалентность не подтверждена), но не ложноположительные.

Улучшение базового ВEq. Основное ограничение базового ВEq низкая полнота: фиксированный набор тактик и односторонний характер поиска приводят к тому, что многие действительно эквивалентные и верные формализации остаются неподтверждёнными. Предлагаемое улучшение направлено именно на повышение полноты при сохранении символической надёжности, а значит, и высокой точности. Оно состоит из трёх компонент.

1. **Двусторонняя проверка.** Вместо одностороннего поиска $T(s_P \mid s_Q, R)$ предпринимается попытка свести как s_P к s_Q , так и s_Q к s_P ; эквивалентность признаётся, если успешна хотя бы одна из двух попыток. Это устраняет зависимость результата от того, какая из формализаций назначена «целью».
2. **Расширенный набор преобразований R .** Базовый набор тактик дополняется преобразованиями, часто возникающими между эквивалентными, но синтаксически различными формулировками: приведением к нормальной форме относительно коммутативности и ассоциативности, согласованием явных и неявных аргументов, перестановкой независимых гипотез и кванторов, а также контролируемым развёртыванием (δ -редукцией) часто встречающихся определений `mathlib` и нормализацией через `simp` по выверенному набору лемм.
3. **Предварительная нормализация.** Перед поиском обе формализации приводятся к канонической форме (нормализация связанных переменных и порядка аргументов), что сокращает пространство поиска и повышает долю случаев, разрешаемых уже на этапе проверки равенства по определению.

Принципиально, что все три компоненты лишь расширяют множество предлагаемых преобразований, тогда как финальную эквивалентность по-прежнему подтверждает ядро Lean. Поэтому повышение полноты не должно достигаться ценой надёжности. Точность остаётся близкой к 100%. Схематично процедура расширенного ВEq записывается так:

1. привести s_P и s_Q к канонической форме (нормализация переменных и аргументов);
2. если $s_P \sim_D s_Q$ — вернуть «эквивалентны»;
3. иначе вычислить $T(s_P \mid s_Q, R)$ с расширенным набором R ; при успехе — вернуть «эквивалентны»;
4. иначе вычислить обратное преобразование $T(s_Q \mid s_P, R)$; при успехе — вернуть «эквивалентны»;
5. иначе вернуть «не эквивалентны»,

где $\sim_D$ — равенство по определению, а $T(\cdot \mid \cdot, R)$ — функция преобразования из (3), в которой каждый кандидат-результат верифицируется ядром Lean.

Таблица 1: Качество символических методов на основе компилятора Lean 4 ($N = 1464$, корректных 486). Метрики в процентах.

Метод	Доля правильных ответов	Точность	Полнота	F_1
typecheck	34,5	33,2	96,1	49,3
BEq (базовый)	77,7	100,0	32,7	49,3
BEq (расширенный)	86,2	99,7	58,6	73,8

Сравнение методов. Результаты трёх символических методов на бенчмарке из раздела 3.2 приведены в таблице 1. Метрики вычислены относительно экспертной метки: точность - доля действительно корректных среди признанных корректными, полнота - доля найденных среди всех корректных, F_1 - их гармоническое среднее.

Картина согласуется с теоретическими ожиданиями и наглядно демонстрирует недостаточность проверки типов. Метод **typecheck** обладает почти максимальной полнотой (96,1%): корректные формализации, как правило, компилируются. Однако его точность катастрофически низка (33,2%): подавляющее большинство некорректных кандидатов также успешно компилируется и ошибочно засчитывается как корректные. Доля правильных ответов 34,5% оказывается даже ниже доли отрицательного класса, то есть как самостоятельная метрика качества проверка типов практически бесполезна и пригодна лишь как предварительный фильтр.

Базовый BEq демонстрирует ровно противоположный профиль: ни одного ложного срабатывания (точность 100%), что подтверждает символическую надёжность подхода. Ядро Lean не подтверждает несуществующих эквивалентностей. Платой за это служит низкая полнота: подтверждается лишь 32,7% корректных формализаций, остальные остаются неподтвержденными из-за ограниченности набора преобразований.

Предложенный расширенный вариант BEq существенно сдвигает баланс в нужную сторону. Полнота возрастает с 32,7% до 58,6%, F_1 - с 49,3% до 73,8%, доля правильных ответов с 77,7% до 86,2%. При этом точность остаётся практически идеальной (99,7%): на всю выборку приходится лишь одно ложное срабатывание, связанное с пограничным случаем чрезмерно агрессивной нормализации. Таким образом, расширенный BEq почти не жертвует ключевым свойством надёжности, но заметно лучше базового варианта улавливает действительно эквивалентные, но синтаксически различные формализации.

Таким образом, среди символических методов проверка типов годится лишь как фильтр компилируемости, а содержательное символическое сравнение задаёт именно BEq. Предложенный расширенный вариант BEq превосходит базовый вариант по всем показателям при сохранении почти нулевой доли ложных срабатываний. Ценой надёжности остаётся ограниченная полнота: более половины корректных формализаций по-прежнему не подтверждаются, поскольку их эквивалентность эталону не выражается короткой цепочкой допустимых преобразований. Это и мотивирует обращение к принципиально иному семейству методов, а именно оценке на основе больших языковых моделей.

3.4 Применение больших языковых моделей к оцениванию

Альтернативный подход использовать большую языковую модель в роли судьи. Модели предъявляется неформальное утверждение, эталонная и кандидатная формализации, и она выносит вердикт о семантическом соответствии. Парадигма «модель как судья» закрепились как практичная и хорошо коррелирующая с экспертной оценкой замена дорогой ручной разметке в широком классе задач [26]. В формальной математике

она уже применяется, в частности, для фильтрации формализаций экспертной итерацией с моделью-судьёй [22]. В отличие от символических методов, судья опирается на семантическое понимание и не ограничен заранее заданным набором преобразований, что потенциально снимает проблему низкой полноты, однако ценой недетерминированности и риска ложноположительных срабатываний. Важно подчеркнуть, что БЯМ при оценке видит эталонную формализацию и именно за счет этого имеет возможность провести сравнения и вынести оценку.

Эксперименты с разными моделям. На том же бенчмарке (раздел 3.2) проверены три современные модели в роли судьи при едином подходе: DeepSeek (`deepseek-v4-pro`), Qwen (`qwen-3.6-plus`) и Gemini (`gemini-3.1-pro-preview`). Результаты приведены в таблице 2.

Таблица 2: Качество методов с использованием больших языковых моделей. Метрики в процентах.

Модель судья	Доля правильных ответов	Точность	Полнота	F_1
DeepSeek (v4-pro)	82,3	71,7	77,2	74,3
Qwen (3.6-plus)	82,6	70,2	82,7	75,9
Gemini (3.1-pro-preview)	84,4	74,2	81,3	77,6

Все три судьи кардинально превосходят символические методы по полноте и F_1 при сохранении умеренной точности. Полнота находится в диапазоне 77–83% против 54% у расширенного BEq. Модели улавливают семантическую эквивалентность в случаях, недоступных короткой цепочке тактик. Платой служит появление ложноположительных срабатываний (точность 70–74%, то есть от 26% до 30% признанных корректными формализаций на деле ошибочны). Именно тот риск, которого лишены символические методы. Модели различаются профилем: DeepSeek наиболее консервативен (наименьшая полнота 77,2% при точности 71,7%), Qwen, напротив, более «снисходителен» (наибольшая полнота 82,7% ценой наименьшей точности 70,2%), а Gemini обеспечивает лучший баланс и одновременно максимальные точность (74,2%), долю правильных ответов (84,4%) и F_1 (77,6%).

Ансамблирование через голосование. Поскольку судьи ошибаются по-разному, естественно объединить их вердикты. Рассмотрены три классические стратегии агрегации трёх голосов: объединение (OR - кандидат корректен, если так считает хотя бы один судья), большинство (Majority - не менее двух из трёх) и единогласие (AND - все три судьи). Результаты приведены в таблице 3.

Таблица 3: Ансамблирование трёх моделей различными правилами. Метрики в процентах.

Стратегия	Доля правильных ответов	Точность	Полнота	F_1
OR (хотя бы один)	72,9	55,8	87,4	68,2
Majority (большинство)	88,0	82,3	81,5	81,9
AND (единогласно)	88,4	90,9	72,2	80,5

Голосование даёт явный и предсказуемый рычаг управления балансом точность / полнота. Объединение (OR) максимизирует полноту (87,4% - выше любого отдельного судьи), но резко снижает точность до 55,8% (и долю правильных ответов до 72,9%).

Единогласие (AND), наоборот, максимизирует точность (90,9% - выше любого отдельного судьи) ценой падения полноты до 72,2%. Голосование большинством занимает промежуточное положение и даёт наиболее сбалансированный профиль (82,3% точности при 81,5% полноты). Характерно, что по F_1 голосование большинством и единогласие превосходят лучшего одиночного судью (Gemini, $F_1 = 77,6\%$): большинство поднимает F_1 до 81,9%, единогласие до 80,5%, тогда как объединение (OR), наоборот, проигрывает по F_1 (68,2%), так как покупает полноту ценой обвала точности. Таким образом, ансамблирование не только улучшает сбалансированную метрику, но и даёт возможность целенаправленно максимизировать определенную метрику. Если приоритет не пропустить ни одной верной формализации, выбирается OR. Если важнее не засчитать ошибочную (высокая точность) AND и для сбалансированной оценки большинство.

3.5 Результаты

Сведём все рассмотренные методы вместе в одну таблицу (таблица 4) и сопоставим их не только по качеству, но и по двум практически важным аспектам: вычислительная сложность / стоимость и проверяемости (детерминированности и невозможности ложноположительных срабатываний).

Таблица 4: Итоговое сравнение методов оценивания. Метрики в процентах. Стоимость и проверяемость оценены качественно.

Метод	Доля	Точность	Полнота	F_1	Стоимость	Детерм.
typecheck	34,5	33,2	96,1	49,3	низкая	да
BEq	77,7	100,0	32,7	49,3	средняя	да
BEq(расширенный)	86,2	99,7	58,6	73,8	средняя	да
Судья (Gemini)	84,4	74,2	81,3	77,6	высокая	нет
Судьи: OR	72,9	55,8	87,4	68,2	оч. высокая	нет
Судьи: Majority	88,0	82,3	81,5	81,9	оч. высокая	нет
Судьи: AND	88,4	90,9	72,2	80,5	оч. высокая	нет

Вычислительная стоимость. Стоимость методов различается на порядки. Проверка типов требует одной компиляции на кандидата и практически бесплатна. Базовый и расширенный BEq более затратны: они выполняют поиск по тактикам и многократную проверку ядром, причём расширенный вариант за счёт двусторонней проверки и расширенного набора преобразований требует больше компиляций, чем базовый вариант. Использование больших языковых моделей для оценки на порядок дороже. Каждый вердикт это запуск БЯМ (заметная задержка, потребность в ускорителях), а ансамбли из трёх моделей умножают эту стоимость втрое. К этому добавляется недетерминированность и неинтерпретируемость.

Метрики для оценки качества моделей. Для измерения и сравнения качества моделей в задаче автоматической формализации уместна комбинация двух взаимодополняющих метрик. В качестве надёжной нижней оценки следует использовать расширенный BEq: его почти нулевая доля ложных срабатываний означает, что каждая подтверждённая им формализация действительно корректна, поэтому соответствующая доля это гарантированная оценка качества снизу, к тому же детерминированная и воспроизводимая. В качестве оценки сверху и для анализа того, что BEq пропускает, полезен ансамбль судей, дающий высокую полноту. Однако на практике именно ансамбль

оказывается неприменим: его вычислительная стоимость втрое выше из-за запуска трёх моделей. При этом свести ансамбль к одному судье тоже нельзя из-за низкой точности (всего 70–74%), при которой до трети засчитанных формализаций ошибочны, а оценка качества оказывается завышенной и вдобавок недетерминированной. Поэтому далее в работе для оценки качества используется только расширенный ВEq. Тем не менее при наличии достаточных вычислительных ресурсов комбинацию судей всё же имеет смысл задействовать как дополнительную оценку сверху, дающую более полную картину того, что расширенный ВEq недооценивает из-за своей неполноты.

Награда в обучение с подкреплением. Для обучения с подкреплением ключевым требованием к награде является её проверяемость (раздел 2.3): именно благодаря однозначному вердикту компилятора обучение с подкреплением естественно ложится на задачу доказательства теорем, тогда как для автоматической формализации надёжного автоматического сигнала до сих пор не было. С этой точки зрения семейства методов ведут себя принципиально по-разному.

Модели судьи, несмотря на высокие F_1 , плохо подходят на роль награды. Они недетерминированы, а их 26–30% ложноположительных срабатываний создают прямую возможность взлома награды: политика в ходе обучения с подкреплением может научиться порождать формализации, убеждающие судью, но семантически неверные. Ровно тот риск применения обучения с подкреплением в неverified доменах, на который указывают авторы статьи [22], прямо называя сигнал судьи размытым и ненадёжным.

Поэтому в качестве награды предпочтителен символический сигнал расширенного ВEq. Его точность, близкая к 100%, означает практически полное отсутствие ложноположительных срабатываний: положительная награда выдаётся только за действительно корректную формализацию, что делает сигнал верифицируемым и устойчивым к взлому, а детерминированность обеспечивает стабильность и воспроизводимость обучения. Платой служит разреженность: из-за неполноты ($recall \approx 59\%$) большая часть корректных формализаций получают нулевую награду, что замедляет обучение, но не вводит его в заблуждение. На практике разумна двухуровневая схема: проверка типов как дешёвый предварительный фильтр компилируемости (отсев заведомо невалидных записей), а поверх неё расширенный ВEq как верифицируемый положительный сигнал смысловой корректности. Кроме для онлайн алгоритмов обучения с подкреплением награда подсчитывается прямо во время обучения при каждом обновлении политики, как результат чем сложнее вычислительно награда, тем дольше будет длиться обучение. Поэтому относительно невысокая стоимость вычисления расширенного ВEq по сравнению с использованием БЯМ является ещё одним преимуществом.

Итог. Таким образом, проверка типов необходима, но как метрика качества несостоятельна. Расширенный ВEq даёт надёжный, детерминированный и проверяемый, хотя и неполный сигнал смысловой корректности и улучшает базовый ВEq по всем показателям. Использование больших языковых моделей обеспечивают наилучшую полноту и F_1 , но недетерминированы, дороги и уязвимы к ложноположительным срабатываниям. Полученные результаты смещают практический выбор в пользу расширенного ВEq как метода по умолчанию во всех сценариях.

4 Задача автоматической формализации

4.1 Постановка задачи

Предыдущая глава была посвящена тому, как оценивать формализацию. Автоматические методы оценивания были систематически сопоставлены с экспертной разметкой, и в качестве метрики корректности был выбран расширенный ВEq. Данная метрика является надёжной и проверяемой, хотя и обладает невысокой полнотой (раздел 3.5). Зафиксировав тем самым критерий, по которому формализация признаётся корректной, перейдём от вопроса оценивания к постановке самой задачи автоматической формализации.

Задача рассматривается только на уровне формулировок, то есть формализация доказательств утверждений остается за рамками данной постановки и работы в целом. По утверждению, записанному на естественном языке, требуется построить эквивалентное ему формальное утверждение на языке Lean 4. Целевым объектом служит именно формальная формулировка теоремы, корректность которой устанавливается при помощи ВEq.

Сохраним обозначения, введённые в разделах 2.2 и 2.3. Обозначим через $x \in \mathcal{X}$ математическое утверждение на естественном языке, а через $y \in \mathcal{Y}$ - формальное утверждение на языке Lean 4, записанное в контексте библиотеки mathlib [4]. Формализацию y будем называть *корректной*, если выбранный в предыдущей главе метод оценивания расширенный ВEq признаёт её эквивалентной эталону:

$$\text{ВEq}(y, y^*) = 1. \quad (16)$$

По построению ВEq положительный вердикт (16) устанавливается ядром Lean, автоматически влечёт прохождение проверки типов и не порождает ложноположительных срабатываний (раздел 3.5).

Целью работы является построение модели / алгоритма

$$f_\theta : \mathcal{X} \rightarrow \mathcal{Y},$$

параметризованной большой языковой моделью с параметрами θ , такой что для произвольного утверждения $x \in \mathcal{X}$ её выход $f_\theta(x)$ является корректной формализацией x в указанном выше смысле. В терминах раздела 2.3 модель задаёт условное распределение $p_\theta(y \mid x)$ и формализация $f_\theta(x)$ получается тем или иным способом декодирования из этого распределения.

Построение и оценка модели опираются на набор пар

$$\mathcal{D} = \{(x_i, y_i^*)\}_{i=1}^N,$$

где $x_i \in \mathcal{X}$ утверждение на естественном языке, а $y_i^* \in \mathcal{Y}$ соответствующая ему эталонная формализация, относительно которой проверяется корректность. Качество формализатора оценивается на отложенной тестовой выборке.

4.2 Данные для оценивания

Оценка проводилась на наборе данных FORMALMATH-LITE тщательно отобранные 425 задач [8]. Этот набор охватывает задачи от школьного олимпиадного до университетского уровня (соответственно 359 и 66 задач) и составлен из нескольких разных источников, что обеспечивает разнообразие по областям математики и стилю формулировок. Каждая задача снабжена неформальным утверждением x_i и эталонной формализацией y_i^* , прошедшей многоступенчатую проверку: семантическую верификацию

несколькими языковыми моделями и финальную проверку людьми экспертами, что делает эталонные формализации надёжной точкой отсчёта. Пример записи из данного набора, приведен на рисунке 8.

УТВЕРЖДЕНИЕ НА ЕСТЕСТВЕННОМ ЯЗЫКЕ Известно, что $\sin A + \sin B = 1$ и $\cos A + \cos B = 3/2$. Найдите значение $\cos(A - B)$. Ответ: $5/8$.
ЗАГОЛОВОК ФАЙЛА <pre>import Mathlib open Real Set open scoped Real</pre>
ЭТАЛОННАЯ ФОРМАЛИЗАЦИЯ <pre>theorem omni_theorem_1599 (A B : ℝ) (h₁ : sin A + sin B = 1) (h₂ : cos A + cos B = 3 / 2) : cos (A - B) = 5 / 8 := by sorry</pre>

Рис. 8: Пример записи из набора FORMALMATH-LITE для оценивания автоматической формализации

По этому набору единообразно оцениваются все рассматриваемые в данной главе подходы: как существующие решения (раздел 4.3), так и обученная далее модель (раздел 4.4). Единая выборка и единая процедура замера делают их показатели напрямую сопоставимыми.

Метод измерения на данном наборе устроен следующим образом. Для каждого утверждения на естественном языке x_i из тестовой выборки оцениваемая модель порождает ровно одну формализацию $\hat{y}_i$ на Lean 4. Каждый кандидат $\hat{y}_i$ характеризуется двумя показателями:

- **typecheck** - доля формализаций, успешно проходящих проверку типов компилятором Lean 4 в контексте библиотеки mathlib.
- **BEq** - доля формализаций, признанных эквивалентными эталону y_i^* по расширенному BEq.

По построению BEq влечёт typecheck: семантически эквивалентная эталону формализация обязана компилироваться, поэтому доля BEq не превосходит долю typecheck. Разрыв между этими двумя величинами измеряет долю формализаций, которые компилируются, но не соответствуют исходному утверждению по смыслу.

4.3 Оценка существующих решений

В этом разделе существующие подходы к автоматической формализации сопоставляются на едином наборе данных и по единой методике оценивания. Цель сравнения понять, какие из доступных решений и за счёт чего дают лучшее качество формализации.

Рассматриваются два качественно разных способа решения задачи большими языковыми моделями (ср. раздел 2.4): применение крупной предобученной модели общего назначения без дообучения и использование модели, специально обученной под задачу автоматической формализацию.

Использование предобученных моделей общего назначения. Первый способ взять предобученную модель общего назначения и применить её к задаче напрямую, без какого-либо дообучения, порождая формализацию прямо из неформального утверждения. Возможность применения данного подхода обусловлена наличием эмерджентных способностей больших языковых моделей [27], заключающихся в появлении новых возможностей по мере масштабирования модели без явного обучения соответствующим задачам.

В качестве такой модели используется DeepSeek-V4-Pro рассуждающая модель семейства DeepSeek-V4 с разрежённой архитектурой «смесь экспертов». Она насчитывает порядка 1,6 трлн параметров, из которых на каждый токен активируется лишь около 49 млрд, и поддерживает контекст до 1 млн токенов. Это одна из сильнейших на сегодня открытых моделей общего назначения. Расчёт здесь на то, что столь мощная модель сформализует утверждение, опираясь лишь на знания, усвоенные при предобучении. Кроме этого рассматривается применение контекстного обучения 2.4. В запрос дополнительно помещаются восемь пар «неформальное утверждение – формализация» из разных областей математики, которые задают модели целевой синтаксис Lean 4 и стиль формальных записей, не изменяя при этом её весов.

Полученные результаты приведены в таблице 5. Несмотря на то, что DeepSeek-V4-Pro является предобученной моделью общего назначения, при помощи нее удастся в 82,8% генерировать синтаксически верный код на Lean 4 и в 30,1% семантически эквивалентный эталонной формализации. Объясняется это скорее всего тем, что в обучающую выборку данной модели так или иначе попадали данные из области формальной математики и в частности формулировки на языке Lean 4. При этом дополнительное использование контекстного обучения сильно повышает способности модели. Модель лучше улавливает синтаксис за счет примеров и поэтому метрика typecheck возрастает с 82,8% до 88,0%, при этом метрика семантической эквивалентности растёт не так сильно с 30,1% до 33,4%.

Таблица 5: Измерения качества автоматической формализации на наборе FORMALMATH-LITE ($N = 425$) предобученных моделей.

Подход	Размер	typecheck, %	BEq, %
DeepSeek-V4-Pro, без примеров	≈ 1,6 трлн	82,8	30,1
DeepSeek-V4-Pro, 8 примеров	≈ 1,6 трлн	88,0	33,4

Специализированные модели. Второй способ использовать модель, специально обученную под автоматическую формализацию. Как показано в обзоре литературы (раздел 2.4), под эту задачу дообучают отдельные и, как правило, заметно меньшие модели, прививая им синтаксис и стиль формального языка непосредственно на этапе обучения. Актуальным представителем таких моделей является Kimina-Autoformalizer-7B специализированная модель с 7 млрд параметров, разработанная Project Numina [22]. Она получена дообучением модели Qwen2.5-Coder-7B-Instruct и переводит утверждения на естественном языке в формальное на Lean 4, оканчивающиеся заглушкой `:= by sorry` (то есть решает в точности задачу формализации, без доказательства). Обучение состояло из инициализирующего дообучения с учителем на крупном корпусе пар, после чего следовала структурированная экспертная итерация. Модель порождала кандидатов, из них отбирались компилирующиеся, а их семантическая корректность оценивалась моделью-судьёй и экспертами, и отобранные пары пополняли обучающую выборку на следующей итерации. Результаты приведены в таблице 6.

Таблица 6: Измерения качества автоматической формализации на наборе FORMALMATH-LITE ($N = 425$) специализированной модели.

Подход	Размер	typecheck, %	BEq, %
Kimina-Autoformalizer-7B	7 млрд	92,2	39,3

Сравнение результатов. Сводные результаты приведены в таблице 7. Наблюдается несколько устойчивых закономерностей.

Первая закономерность касается разрыва между двумя метриками. Доля семантически верных формализаций оказывается примерно на 53 процентных пункта ниже доли компилируемых. Даже среди записей, прошедших проверку типов, эталону эквивалентны лишь 36,4% у DeepSeek-V4-Pro без примеров, 38,0% в режиме контекстного обучения и 42,6% у Kimina-Autoformalizer-7B. То есть ни один из подходов не приближается к надёжной формализации, а компилируемость сама по себе систематически переоценивает их качество. Это ещё раз оправдывает выбор BEq как основной метрики (раздел 3.5).

Вторая закономерность — превосходство специализированной модели над универсальной. Kimina-Autoformalizer-7B опережает лучший режим DeepSeek-V4-Pro (few-shot) на 4,2 пункта по typecheck и на 5,9 пункта по BEq, причём больший отрыв приходится именно на семантическую метрику. При этом Kimina несопоставимо меньше: 7 млрд параметров против $\approx 1,6$ трлн у DeepSeek-V4-Pro.

Таблица 7: Сравнение существующих подходов к автоматической формализации на наборе FORMALMATH-LITE ($N = 425$).

Подход	Размер	typecheck, %	BEq, %
DeepSeek-V4-Pro, zero-shot	$\approx 1,6$ трлн	82,8	30,1
DeepSeek-V4-Pro, few-shot (8)	$\approx 1,6$ трлн	88,0	33,4
Kimina-Autoformalizer-7B	7 млрд	92,2	39,3

Выводы. Специализированная модель Kimina-Autoformalizer-7B, будучи на порядки меньше модели общего назначения, тем не менее решает задачу формализации точнее и при этом вычислительно дешевле и быстрее. Это показывает, что в задаче автоматической формализации перспективным является не масштабирование универсальных моделей, а развитие отдельных, специализированных под формализацию моделей.

4.4 Обучение модели

Сравнение существующих решений показало, что наиболее перспективным направлением является не масштабирование универсальной модели, а дообучение компактной специализированной модели под задачу автоматической формализации. Сильнейший из рассмотренных подходов, Kimina-Autoformalizer-7B, обязан своим качеством прежде всего этапу дообучения с учителем на тщательно отобранном корпусе пар «неформальное утверждение - формализация». Это согласуется с наблюдением из обзора литературы (раздел 2.4): на этапе обучения формализатора ключевую роль играет не столько объём, сколько качество обучающих данных. Опираясь на этот вывод, в настоящем разделе обучается собственная модель формализатора. Основной акцент

делается на чистоте обучающего корпуса, а сам процесс ограничивается одним проходом дообучения с учителем, без дорогостоящих итераций экспертного отбора, которыми пользовались авторы работы [22].

Данные для обучения. Обучающий корпус собран из нескольких открытых наборов данных по формализации на Lean 4 и приведён к единому формату. Источниками послужили:

- COMBIBENCH [25] - набор комбинаторных задач.
- LEAN-WORKBOOK [15] - масштабный корпус задач соревновательного уровня, формализованных из условий на естественном языке;
- PROOFNET [7] - набор утверждений университетского уровня из классических учебников
- FORMALMATH [8] - крупный верифицированный набор формальных утверждений от школьного олимпиадного до университетского уровня;
- MINIF2F [6] - стандартный набор задач олимпиадного уровня;

Все источники приведены к общей схеме из неформального утверждения x_i , заголовка формализации (`import/open`-директивы) и тела теоремы, а концовка каждой формализации нормализована к единому виду `:= by sorry`, то есть к задаче формализации формулировки без доказательства.

Поскольку качество эталонных формализаций в исходных наборах неоднородно, к собранному корпусу применялась двухступенчатая фильтрация по семантической корректности. На первом шаге автоматически отбирались пары, проходящие проверку расширенным BEq - тем же критерием эквивалентности эталону, что выбран в главе об оценивании. Пары, не прошедшие автоматическую проверку, не отбрасывались сразу, а передавались на дополнительную оценку нескольким языковым моделям-судьям; пары, которые судьи единогласно признавали семантически неверными формализациями, исключались из корпуса, остальные сохранялись. Такой отбор позволяет удалить заведомо ошибочные пары, не теряя при этом корректные формализации, которые BEq не распознаёт из-за его неполноты (раздел 3.5).

Итоговый очищенный корпус опубликован в открытом доступе¹ и насчитывает около 30 тыс. пар. Из него случайным образом выделена отложенная валидационная выборка из 400 пар, не пересекающаяся с обучающей. На ней по ходу обучения отслеживались как функция потерь, так и метрики (typecheck и расширенный BEq). Валидационная выборка получена разделением обучающего корпуса и потому отлична по распределению от тестового набора FORMALMATH-LITE (раздел 4.2), на котором проводится финальное сравнение с существующими решениями. Значения метрик на валидации и на тесте поэтому не следует сопоставлять напрямую.

Метод обучения Дообучение проводилось в режиме стандартного дообучения с учителем. Каждой паре (x_i, y_i^*) из обучающей выборки сопоставляется текстовый запрос (промпт для БЯМ) c_i и целевой ответ. Запрос c_i строился следующим образом: системная инструкция задавала роль эксперта по математике и Lean 4, а пользовательская часть содержала неформальное утверждение, фиксированное имя теоремы и требуемый заголовок. Целевым ответом служил единый `lean4` блок с заголовком и утверждением.

¹<https://huggingface.co/datasets/AlexVav01/autoformalization-clean>

После токенизации он образует последовательность токенов $w_i(w_{i,1}, \dots, w_{i,T_i})$, кодирующую эталонную формализацию y_i^* . Пример запроса и целевого ответа приведен на рисунке 9.

Системный промпт You are an expert in mathematics and Lean 4.
Пользовательский промпт (запрос) Please autoformalize the following problem in Lean 4 with a header. Use the following theorem names: my_favorite_theorem. It is known that $\sin A + \sin B = 1$ and $\cos A + \cos B = 3/2$. Find the value of $\cos(A - B)$. Answer: $5/8$. Your code should start with: <pre> ```lean4 import Mathlib open Real Set open scoped Real ``` </pre>
Целевой ответ <pre> ```lean4 import Mathlib open Real Set open scoped Real theorem my_favorite_theorem (A B : ℝ) (h₁ : sin A + sin B = 1) (h₂ : cos A + cos B = 3 / 2) : cos (A - B) = 5 / 8 := by sorry ``` </pre>

Рис. 9: Пример запроса в модель и целевого ответа для обучения. Функция потерь считается только на токенах целевого ответа.

Перейдем к описанию алгоритма обучения. Языковая модель с параметрами θ задаёт условное распределение над ответом авторегрессионно, как произведение распределений над очередным токеном при условии запроса и уже порождённого префикса:

$$p_{\theta}(w_i \mid c_i) = \prod_{t=1}^{T_i} p_{\theta}(w_{i,t} \mid c_i, w_{i,<t}). \quad (17)$$

Дообучение с учителем максимизирует правдоподобие эталонных ответов, что эквивалентно минимизации усреднённой по токенам кросс-энтропии (отрицательного лог-правдоподобия). Существенно, что функция потерь считается только на токенах ответа w_i , а токены запроса c_i маскируются и в неё не входят, поэтому модель учится порождать формализацию, но не воспроизводить условие. Решаемая задача оптимизации записывается как

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta), \quad \mathcal{L}(\theta) = - \frac{1}{\sum_i T_i} \sum_i \sum_{t=1}^{T_i} \log p_{\theta}(w_{i,t} \mid c_i, w_{i,<t}), \quad (18)$$

где суммирование ведётся по всем парам обучающей выборки и всем токенам их ответов.

Целевая функция (18) минимизируется стохастически по мини-батчам алгоритмом AdamW [28]. На шаге t выбирается мини-батч $\mathcal{B}_t$ и вычисляется оценка градиента

$g_t = \nabla_{\theta} \mathcal{L}_{B_t}(\theta_{t-1})$. Во время обучения эффективный размер мини-батча составлял 8 примеров. AdamW поддерживает экспоненциально сглаженные оценки первого и второго моментов градиента, m_t и v_t , и обновляет параметры по их скорректированным на смещение значениям:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, & \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t, & \hat{v}_t &= \frac{v_t}{1 - \beta_2^t}, \\ \theta_t &= \theta_{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right), \end{aligned} \quad (19)$$

где все операции над векторами поэлементны. Использовались стандартные значения $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\epsilon = 10^{-8}$. Параметр η_t задавался линейным ростом на первых $T_w = [0,03 T]$ шагах от нуля до пикового значения $\eta_{\max} = 3 \cdot 10^{-5}$ и последующим косинусным затуханием до нуля:

$$\eta_t = \eta_{\max} \cdot \frac{1}{2} \left(1 + \cos \pi \frac{t - T_w}{T - T_w} \right), \quad t > T_w, \quad (20)$$

где $T \approx 7,7 \cdot 10^3$ полное число шагов оптимизации, соответствующее 2 эпохам прохода по обучающей выборке.

Каждые 800 шагов обучения на валидационной выборке вычислялись функция потерь, а также `typescheck` и расширенный BEq. Для последних модель в детерминированном режиме («жадное» декодирование) порождала по одной формализации на задачу, которые затем проверялись ядром Lean.

Динамика обучения приведена на рисунках 10 и 11. Функция потерь на обучающей выборке монотонно убывает с 0,39 до примерно 0,03, с характерным дополнительным спадом на границе второй эпохи. Потери на валидации, напротив, выходят на плато около 0,10 уже к 4000-му шагу и далее практически не меняются. Дальнейшее снижение обучающей функции потерь отражает переобучение, не улучшая обобщающие способности модели в данной задаче. Это плато согласуется с поведением содержательных метрик `typescheck` и расширенного BEq (рис. 11): обе быстро растут в первой половине обучения и стабилизируются во второй, достигая на валидации примерно 88% по `typescheck` и 33% по BEq⁺. Дальнейшее обучение не приносит выигрыша по семантической метрике, поэтому для финального замера использовалась сохраненная версия модели из области выхода метрик на плато.

Результаты обучения и выводы. По завершении обучения модель была замерена на том же тестовом наборе FORMALMATH-LITE ($N = 425$) и по той же процедуре, что и существующие решения (раздел 4.3), в режиме одной генерации на задачу. Веса обученной модели опубликованы в открытом доступе.² Полученная модель проходит проверку типов на 89,9% задач и является семантически эквивалентной эталону по расширенному BEq на 42,8% задач. При этом среди формализаций, прошедших проверку типов, эталону эквивалентны 47,6%, то есть содержательно верной оказывается почти половина компилируемых записей. Сводное сравнение обученной модели с существующими решениями приведено в таблице 8.

Главный результат состоит в том, что обученная модель превосходит сильнейшее из существующих решений, Kimina-Autoformalizer-7B, по ключевой семантической метрике: 42,8% против 39,3% по расширенному BEq (+3,5 процентных пункта). По

²<https://huggingface.co/AlexVav01/lean4-autoformalizer-sft>

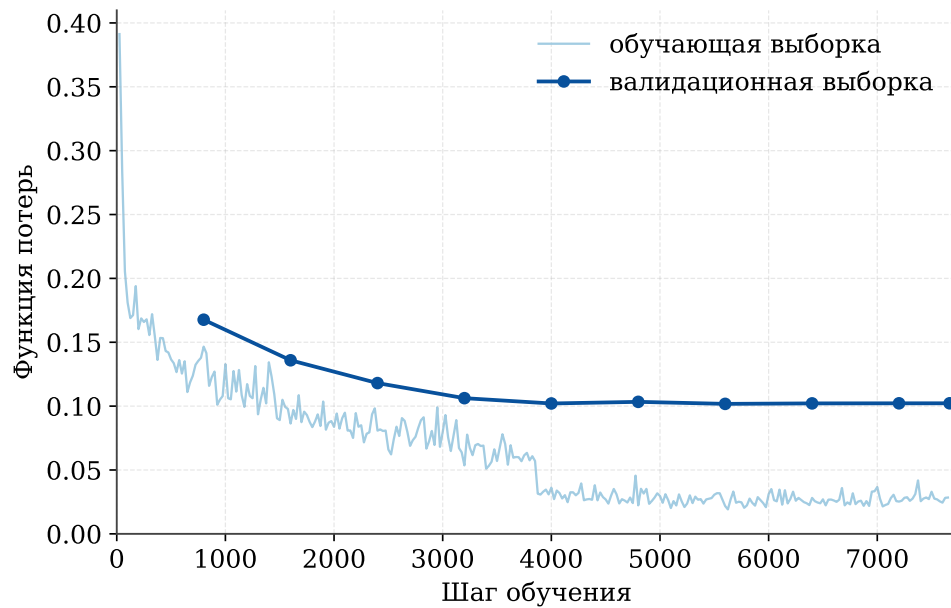


Рис. 10: Динамика функции потерь в ходе обучения с учителем: потери на обучающей выборке (запись каждые 25 шагов) и на отложенной валидационной выборке (каждые 800 шагов).

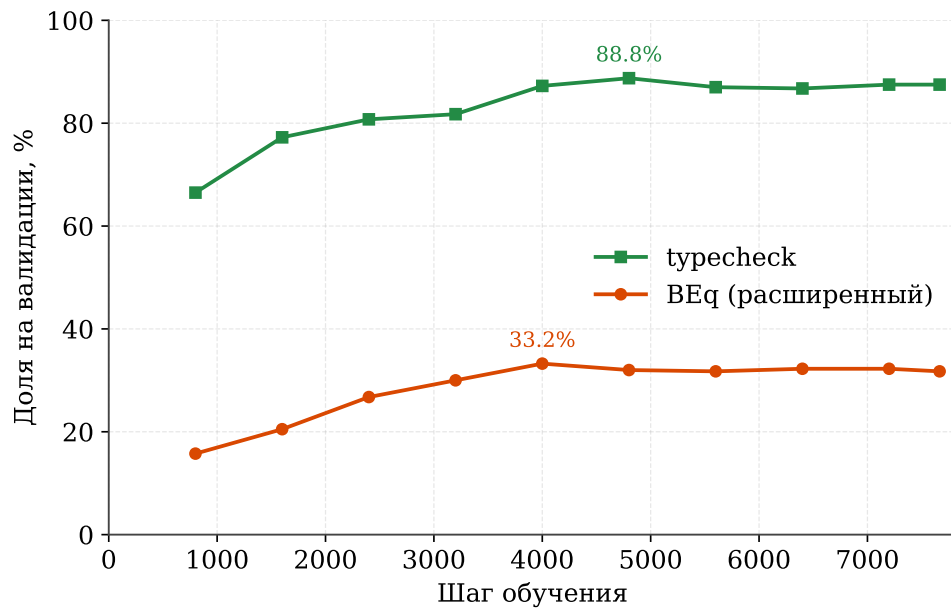


Рис. 11: Метрики на валидационной выборке в ходе дообучения: доля формализаций, проходящих проверку типов (typecheck), и доля эквивалентных эталону по расширенному BEq.

проверке типов модель уступает Kimina (89,9% против 92,2%), однако, как показано в разделе 4.3, именно семантическая метрика, а не компилируемость, отражает реальное качество формализации. При этом обе модели имеют одинаковый размер (7 млрд параметров) и одну и ту же базовую модель Qwen2.5-Coder-7B-Instruct, а модель общего назначения DeepSeek-V4-Pro, на три порядка более крупная, уступает обоим специализированным моделям.

Поскольку базовая модель и её размер совпадают с Kimina, наблюдаемое преимущество по BEq является следствием использования более крупного и качественного

Таблица 8: Сравнение обученной модели с существующими подходами на наборе FORMALMATH-LITE ($N = 425$).

Подход	Размер	typecheck, %	BEq, %
DeepSeek-V4-Pro, zero-shot	$\approx 1,6$ трлн	82,8	30,1
DeepSeek-V4-Pro, few-shot (8)	$\approx 1,6$ трлн	88,0	33,4
Kimina-Autoformalizer-7B	7 млрд	92,2	39,3
lean4-autoformalizer-sft	7 млрд	89,9	42,8

набора данных. Отфильтрованный по семантической корректности корпус прививает модели более точное смысловое соответствие формализаций исходным утверждениям. Особенно показательно, что этот результат достигнут одним проходом дообучения с учителем, без многоступенчатой экспертной итерации, за счёт которой совершенствовалась Kimina. Это говорит о том, что качество обучающих данных способно дать больший выигрыш, чем наращивание числа итераций отбора, и что у предложенного подхода остаётся значительный запас для дальнейшего улучшения.

5 Заключение

Результаты Проведённое исследование было нацелено на автоматическую формализацию формулировок математических теорем на язык Lean 4 средствами больших языковых моделей и выстраивалось вокруг двух тесно связанных вопросов: как надёжно и автоматически оценивать формализацию и как, опираясь на такую оценку, строить формализующую модель. К основным выводам работы можно отнести следующее.

1. Проверка типов, несмотря на распространённость, как самостоятельная метрика качества несостоятельна. Она почти не отличает компилируемую, но семантически неверную формализацию от верной. Высокой точностью обладает лишь символический критерий семейства BEq [11], а оценка с помощью моделей-судей достигает высокой полноты ценой недетерминированности и ложноположительных срабатываний.
2. Предложенное расширение BEq (двусторонняя проверка, расширенный набор семантически сохраняющих преобразований и предварительная нормализация) снимает главный недостаток базового варианта, а именно низкую полноту, поднимая её с 32,7 % до 58,6 %. Тем самым удаётся получить детерминированный и проверяемый сигнал смысловой корректности.
3. Обученная на отфильтрованном по семантической корректности корпусе (около 30 тыс. пар) модель с 7 млрд параметров достигает 42,8 % по расширенному BEq и превосходит по этой метрике одно из сильнейших существующих решений, Kimina-Autoformalizer-7B [12] (39,3 %), уже одним проходом дообучения с учителем, без многоступенчатой экспертной итерации. Корпус и веса модели выложены в открытый доступ.

Ограничения Несмотря на достигнутое улучшение, расширенный BEq сохраняет ограниченную полноту (около 59 %). Значительная часть действительно корректных формализаций не подтверждаются, поскольку их эквивалентность эталону не выражается короткой цепочкой допустимых преобразований. Поэтому он даёт надёжную, но заниженную оценку качества (гарантированную оценку снизу, а не точное значение). Обученная модель построена одним проходом дообучения с учителем, без многоступенчатой экспертной итерации и без обучения с подкреплением.

Дальнейшая работа Из полученных результатов вытекает несколько естественных направлений развития.

Во-первых, перспективным выглядит обучение рассуждающей модели формализации: модель сначала проводит промежуточное рассуждение на естественном языке (например, внутри блока `<think>`), согласуя неформальные и формальные определения, и лишь затем порождает формализацию на Lean 4. Такой приём уже принёс заметный выигрыш в смежной задаче доказательства теорем [12, 23] и в целом на сложных математических задачах [20], тогда как для автоматической формализации он практически не применялся, хотя основная масса ошибок здесь объясняется не синтаксисом, а рассогласованием определений и неявными предположениями. Главным препятствием остаётся отсутствие обучающих корпусов с рассуждениями именно для задачи формализации, что делает их построение отдельным самостоятельным направлением.

Во-вторых, разработанный расширенный BEq может быть использован в качестве награды при обучении с подкреплением, что в настоящей работе подробно не исследовалось. Именно отсутствие надёжного проверяемого сигнала эквивалентности до

сих пор оставалось главным препятствием для переноса обучения с подкреплением, и в частности алгоритма GRPO [24], с задачи доказательства теорем на автоматическую формализацию. Близкая к 100 % точность расширенного BEq и его детерминированность делают такой сигнал устойчивым к взлому награды.

В-третьих, в качестве базовой модели в работе использовалась Qwen2.5-Coder-7B-Instruct к настоящему времени уже сравнительно устаревшая модель. Поскольку предложенный подход не привязан к конкретной базовой модели и сводится к дообучению на очищенном корпусе, его прямое перенесение на более новые и крупные базовые модели способно дать дополнительный прирост качества формализации без изменения самой методики.

Наконец, поскольку достигнутое преимущество модели обусловлено прежде всего качеством обучающих данных, дальнейшее повышение чистоты и расширение корпуса, а также добавление экспертных итераций отбора способны дать дополнительный прирост. В итоге, предложенная метрика оценивания и обученная модель автоматической формализации закладывают методологическую основу для перехода к обучению формализаторов при помощи обучения с подкреплением с автоматической проверяемой наградой и к дальнейшему развитию формализации математики.

Список литературы

- [1] *Nipkow, Tobias*. Isabelle/HOL: A Proof Assistant for Higher-Order Logic / Tobias Nipkow, Lawrence C. Paulson, Markus Wenzel. — Springer, 2002. — Vol. 2283 of *Lecture Notes in Computer Science*.
- [2] *Bertot, Yves*. Interactive Theorem Proving and Program Development: Coq'Art: The Calculus of Inductive Constructions / Yves Bertot, Pierre Castéran. — Springer Science & Business Media, 2013.
- [3] *de Moura, Leonardo*. The Lean 4 Theorem Prover and Programming Language / Leonardo de Moura, Sebastian Ullrich // *Automated Deduction – CADE 28*. — Springer, 2021. — Pp. 625–635.
- [4] *The mathlib Community*. The Lean Mathematical Library / The mathlib Community // *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. — 2020. — Pp. 367–381.
- [5] Attention is All You Need / Ashish Vaswani, Noam Shazeer, Niki Parmar et al. // *Advances in Neural Information Processing Systems (NeurIPS 2017)*. — Vol. 30. — 2017.
- [6] *Zheng, Kunhao*. miniF2F: A Cross-System Benchmark for Formal Olympiad-Level Mathematics / Kunhao Zheng, Jesse Michael Han, Stanislas Polu // *International Conference on Learning Representations (ICLR 2022)*. — 2022.
- [7] ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics / Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf et al. // *arXiv preprint arXiv:2302.12433*. — 2023.
- [8] FormalMATH: Benchmarking Formal Mathematical Reasoning of Large Language Models / Zhouliang Yu, Ruotian Peng, Keyi Ding et al. // *arXiv preprint arXiv:2505.02735*. — 2025.
- [9] Autoformalization with Large Language Models / Yuhuai Wu, Albert Q. Jiang, Wenda Li et al. // *Advances in Neural Information Processing Systems (NeurIPS 2022)*. — Vol. 35. — 2022.
- [10] DeepSeek-Prover: Advancing Theorem Proving in LLMs through Large-Scale Synthetic Data / Huajian Xin, Daya Guo, Zhihong Shao et al. // *arXiv preprint arXiv:2405.14333*. — 2024.
- [11] Rethinking and Improving Autoformalization: Towards a Faithful Metric and a Dependency Retrieval-Based Approach / Qi Liu, Xinhao Zheng, Xudong Lu et al. // *International Conference on Learning Representations (ICLR)*. — 2025.
- [12] *Numina*. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning / Numina, Kimi Team // *arXiv preprint arXiv:2504.11354*. — 2025.
- [13] BLEU: a Method for Automatic Evaluation of Machine Translation / Kishore Papineni, Salim Roukos, Todd Ward, Wei-Jing Zhu // *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. — 2002. — Pp. 311–318.

- [14] Wang, Qingxiang. First Experiments with Neural Translation of Informal to Formal Mathematics / Qingxiang Wang, Cezary Kaliszyk, Josef Urban // Intelligent Computer Mathematics (CICM 2018). — Vol. 11006 of *Lecture Notes in Computer Science*. — Springer, 2018. — Pp. 255–270.
- [15] Lean Workbook: A Large-Scale Lean Problem Set Formalized from Natural Language Math Problems / Huaiyuan Ying, Zijian Wu, Yihan Geng et al. // Advances in Neural Information Processing Systems (NeurIPS 2024). — 2024.
- [16] Process-Driven Autoformalization in Lean 4 / Jianqiao Lu, Yingjia Wan, Zhengying Liu et al. // *arXiv preprint arXiv:2406.01940*. — 2024.
- [17] Luong, Minh-Thang. Effective Approaches to Attention-based Neural Machine Translation / Minh-Thang Luong, Hieu Pham, Christopher D. Manning // Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing (EMNLP). — Association for Computational Linguistics, 2015. — Pp. 1412–1421.
- [18] Exploration of Neural Machine Translation in Autoformalization of Mathematics in Mizar / Qingxiang Wang, Chad Brown, Cezary Kaliszyk, Josef Urban // Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020). — ACM, 2020. — Pp. 85–98.
- [19] Language Models are Few-Shot Learners / Tom B. Brown, Benjamin Mann, Nick Ryder et al. // Advances in Neural Information Processing Systems (NeurIPS 2020). — Vol. 33. — 2020. — Pp. 1877–1901.
- [20] Chain-of-Thought Prompting Elicits Reasoning in Large Language Models / Jason Wei, Xuezhi Wang, Dale Schuurmans et al. // Advances in Neural Information Processing Systems (NeurIPS 2022). — Vol. 35. — 2022. — Pp. 24824–24837.
- [21] Jiang, Albert Q. Multilingual Mathematical Autoformalization / Albert Q. Jiang, Wenda Li, Mateja Jamnik // *arXiv preprint arXiv:2311.03755*. — 2023.
- [22] Numina. Kimina-Prover Preview: Towards Large Formal Reasoning Models with Reinforcement Learning / Numina, Kimi Team // *arXiv preprint arXiv:2504.11354*. — 2025.
- [23] DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning / Daya Guo, Dejian Yang, Haowei Zhang et al. // *arXiv preprint arXiv:2501.12948*. — 2025.
- [24] DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models / Zhihong Shao, Peiyi Wang, Qihao Zhu et al. // *arXiv preprint arXiv:2402.03300*. — 2024.
- [25] Liu, Junqi. CombiBench: Benchmarking LLM Capability for Combinatorial Mathematics / Junqi Liu et al. // *arXiv preprint arXiv:2505.03171*. — 2025.
- [26] Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena / Lianmin Zheng, Wei-Lin Chiang, Ying Sheng et al. // Advances in Neural Information Processing Systems (NeurIPS). — 2023. — arXiv:2306.05685.
- [27] Emergent Abilities of Large Language Models / Jason Wei, Yi Tay, Rishi Bommasani et al. // *Transactions on Machine Learning Research*. — 2022.

- [28] *Loshchilov, Ilya*. Decoupled Weight Decay Regularization / Ilya Loshchilov, Frank Hutter // International Conference on Learning Representations (ICLR 2019). — 2019.